جامعة مولاي إسماعيل
UNIVERSITÉ MOULAY ISMAÏL

الكلية المتعددة التخصصات
FACULTÉ POLYDISCIPLINAIRE

Année Universitaire : 2024-2025

Programmation C 1

Licence Digitalisation - Tronc Commun Informatique Appliquée • Semestre 1

Pr. Mounir Hanine

Plan du cours : programmation en langage C

Chapitre 0
Architecture de l'ordinateur et systèmes de numération

Chapitre 1
Introduction au langage C

Chapitre 2
Les opérateurs standards

Chapitre 3
Les conversions de type

Chapitre 4
Lire et écrire des données

Chapitre 5
Les structures alternatives

Chapitre 6
Les structures répétitives

Chapitre 7
Les tableaux

Chapitre 8
Les chaînes de caractères

Chapitre 9
Les fonctions

Architecture d’un ordinateur et système de numération

Introduction

Cette section introduit les bases de l'informatique, des ordinateurs et de la programmation, et offre un aperçu du traitement automatique de l'information ainsi que des systèmes de numération et de codage.

Architecture et composants matériels de l’ordinateur

Un système informatique, en particulier un ordinateur, se compose de deux grandes parties :**La partie matérielle (ou hardware),** qui regroupe les différents composants physiques de l’appareil. **La partie logicielle (ou software**), qui inclut les programmes, applications et systèmes d’exploitation permettant d’utiliser le matériel pour accomplir les tâches demandées.



Matérielle ou Hardware



Logicielle ou Software

Partie matériel (ou hardware)

Les périphériques

Les périphériques entourent le PC et se répartissent en deux grandes catégories :
Les périphériques d'entrée, qui fournissent des informations au système, comme une souris, un clavier ou une manette de jeu.
Les périphériques de sortie, qui restituent des informations, comme une imprimante ou un écran.



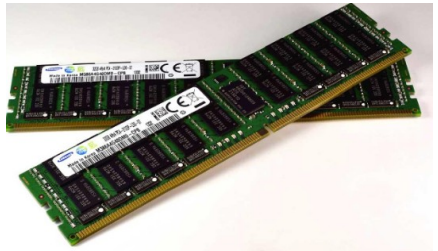
Le processeur (CPU)

Le processeur, ou CPU (Central Processing Unit), est le cerveau de l'ordinateur. Il s'occupe de traiter les instructions des programmes, de les décoder, et de réaliser les calculs nécessaires pour donner les résultats souhaités.



La mémoire vive (RAM : Random Access Memory)

La RAM est une mémoire temporaire qui stocke les données nécessaires au processeur pour exécuter les programmes en cours, permettant à l'ordinateur de gérer plusieurs tâches rapidement sans accéder au disque dur.



Architecture d'un ordinateur et système de numération

Partie matériel

Disque dur

Le disque dur sert à stocker toutes les données de manière permanente **sous forme de fichiers binaires**, même quand l'ordinateur est éteint. Il garde les fichiers, les programmes, le système d'exploitation, et tout ce qui doit être conservé à long terme. Quand on allume l'ordinateur, les informations nécessaires sont chargées depuis le disque dur vers la RAM pour que le processeur puisse y accéder rapidement.

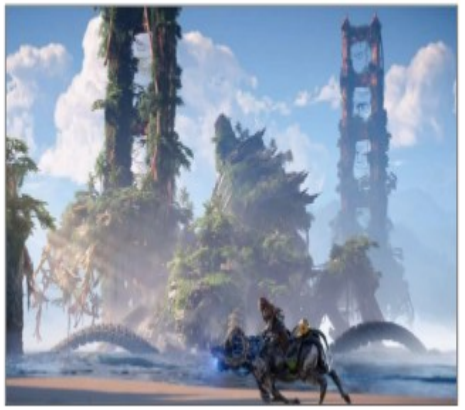


Les deux grandes familles de disques durs sont les **HDD** (**Hard Disk Drive**) et les **SSD** (**Solid State Drive**).
Les **HDD**, basés sur des plateaux magnétiques, offrent de grandes capacités de stockage à un coût inférieur, mais sont plus lents et sensibles aux chocs.
Les **SSD**, utilisant de la mémoire flash, sont beaucoup plus rapides et durables, mais coûtent généralement plus cher par gigaoctet.

Partie matériel

Carte graphique (GPU Graphics Processing Unit)





La carte graphique, ou GPU, décharge le processeur en prenant en charge certains calculs, en particulier ceux liés à la **3D** et au traitement d'images, comme pour les jeux vidéo.

La carte graphique n'est pas indispensable pour tous les ordinateurs ; un GPU dédié est surtout utile pour les tâches exigeantes en graphisme, comme les jeux vidéo, le montage vidéo ou la modélisation 3D. Pour les usages basiques (bureautique, navigation web), le processeur avec un GPU intégré suffit généralement.

Le BIOS



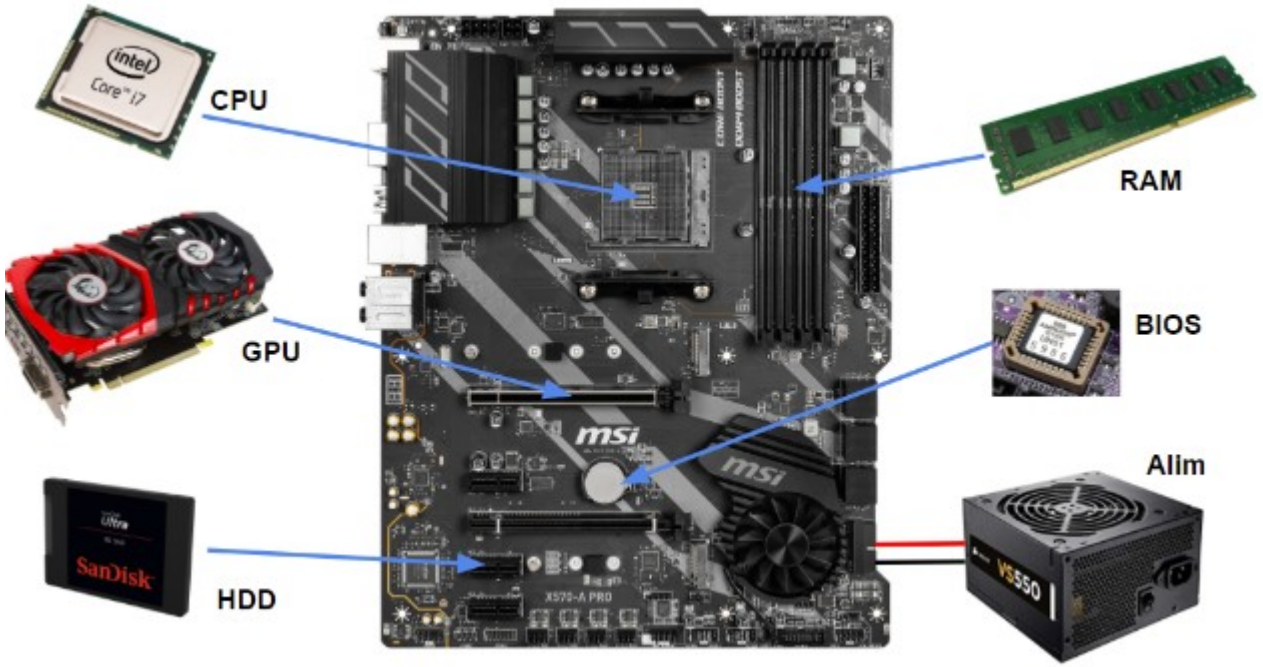


Le BIOS est un composant essentiel pour démarrer un ordinateur. Il contient un programme qui initialise les composants matériels et charge le système d'exploitation.

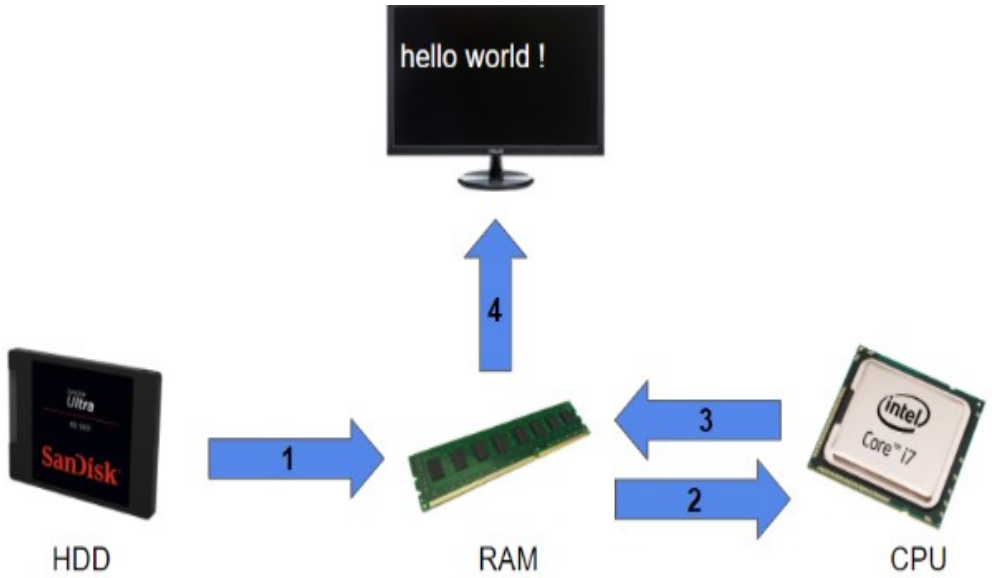
Partie matériel

La carte mère

La carte mère est le cœur de l'ordinateur, sur lequel sont connectés tous les composants essentiels, comme le processeur (CPU : Central Processing Unit), la mémoire vive (RAM), le disque dur, et les cartes graphiques ou sonores. Elle assure l’alimentation et la communication entre ces composants grâce à des bus.



Exécution d’un programme



Le programme est chargé dans la mémoire RAM depuis le disque dur pour une exécution rapide.
Le processeur exécute les instructions, en les récupérant et en les décodant, et en effectuant les opérations nécessaires.
Les résultats sont stockés temporairement en RAM et ensuite affichés sur l'écran à travers les circuits de sortie.
Toutes ces opérations sont effectuées en binaire, où chaque donnée et instruction est représentée par des zéros et des uns.

Systèmes de Numération

Pourquoi bien comprendre les systèmes de numération ?



Les systèmes de numération sont essentiels en informatique, car les ordinateurs ne comprennent que le langage binaire, composé de 0 et 1. Chaque donnée qu'il s'agisse de texte, d'image ou de son est converti en binaire pour être traitée. En programmation, notamment en langage C, il est courant de travailler avec différentes bases comme le binaire, l'octal et l'hexadécimal. Ces bases permettent de manipuler les données plus facilement et de mieux comprendre le fonctionnement des algorithmes. De plus, la gestion de la mémoire et l'optimisation du code nécessitent souvent des opérations sur les bits, comme dans le cas des options ou des permissions de fichiers. Par ailleurs, les caractères sont représentés par des codes numériques (comme ASCII), ce qui rend ces connaissances indispensables pour manipuler différents types de données

Systèmes de Numération

Lien avec l'informatique : Comment les ordinateurs manipulent les nombres et le texte en binaire

Les ordinateurs fonctionnent exclusivement en binaire, un système de numération qui utilise deux symboles : **0 et 1**. Voici comment ils manipulent les nombres et le texte :

A- Représentation des Nombres

Nombres entiers : Ils sont directement convertis en une suite de bits **(ex. : 5 en binaire = 101**).

Nombres négatifs : Utilisation du complément à deux pour représenter les valeurs (ex. : **-5 devient 11111011** sur 8 bits).

Nombres à virgule flottante : Les ordinateurs utilisent la norme **IEEE 754** pour représenter ces nombres, en séparant la **mantisse** et **l'exposant**.

B- Représentation du texte

Chaque caractère est converti en un code numérique grâce au système **ASCII** ou **Unicode**. Exemple : La lettre **'A'** correspond à **65** en décimal, soit **01000001** en binaire.

C- Traitement des Données en Binaire

Les opérations mathématiques (addition, multiplication, etc.) sont effectuées directement sur les bits.

D- Stockage et Transmission des Informations

Les fichiers et programmes sont stockés sous forme binaire dans la mémoire de l'ordinateur.

Les réseaux et périphériques transmettent également les informations en bits (séries de 0 et 1).

Systèmes de Numération

Définitions des principaux systèmes de numération

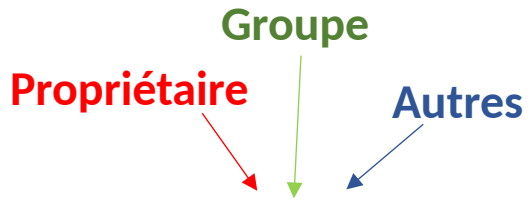
1. Système décimal (base 10)
Description : C'est le système que nous utilisons au quotidien.
Chiffres disponibles : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Principe : Chaque position d'un chiffre correspond à une puissance de 10.
Exemple :
Le nombre **375** se décompose ainsi : $375=(3\times10^2)+(7\times10^1)+(5\times10^0)$

---

2. Système binaire (base 2)
Description : Utilisé par les ordinateurs car ils ne manipulent que deux états : **0** ou **1**.
Chiffres disponibles : 0, 1.
Principe : Chaque position d'un bit représente une puissance de 2.
Exemple : Le nombre **1011** en binaire correspond à : $1011_2=(1\times2^3)+(0\times2^2)+(1\times2^1)+(1\times2^0)= 11$ en décimal

Systèmes de Numération

Définitions des principaux systèmes de numération



3. Système octal (base 8)

Description : Parfois utilisé en programmation, notamment dans les systèmes de type UNIX (Ex: 754).
Chiffres disponibles : 0, 1, 2, 3, 4, 5, 6, 7.
Principe : Chaque position représente une puissance de 8.
Exemple :
Le nombre **17** en octal se décompose en : $17_8 = 1\times8^1 + 7\times8^0 = 15$ **en décimal.**

Lecture (r) : Valeur 4
Écriture (w) : Valeur 2
Exécution (x) : Valeur 1

Lien avec le binaire : Chaque chiffre octal correspond à **3 bits** en binaire exemple : **7 en octal = 111 en binaire**.

4. Système hexadécimal (base 16)

Description : Utilisé pour représenter de grandes quantités de manière compacte (ex. : adresses mémoire).
Chiffres disponibles : **0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F**. **Avec A= 10; B=11; C=12; D=13; E=14 et F=15**
Principe : Chaque position représente une puissance de **16**.
Exemple : Le nombre **2F** en hexadécimal correspond à : $2\times16^1 + 15\times16^0 = (47)_{10}$

Lien avec le binaire : Chaque chiffre hexadécimal correspond à **4 bits** en binaire (ex. : F = 1111).

$(2F)_{16}$ = **0010 1111**

Systèmes de Numération

Lien entre les systèmes : Conversion entre bases

1. Conversion du binaire vers le décimal

Méthode : On multiplie chaque bit par sa puissance de 2 et on additionne.

Exemple : **1011** en binaire = $1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0 = $**11** en décimal

---

2. Conversion du décimal vers le binaire

Méthode : On divise le nombre par 2 en gardant les restes jusqu'à atteindre 0, ensuite on Lit les restes de bas en haut.

Exemple :

$13_{10} \rightarrow 13 \div 2 = 6$ (reste **1**)

$6 \div 2 = 3$ (reste **0**)

$3 \div 2 = 1$ (reste **1**)

$1 \div 2 = 0$ (reste **1**)



Résultat : $13_{10} = 1101_2$

Systèmes de Numération

Conversion du binaire vers l'hexadécimal

Méthode : On groupe les bits par 4, de droite à gauche, et on remplace chaque groupe par son équivalent hexadécimal.

Exemple : $11100110_2 \rightarrow 1110\ 0110 \rightarrow (E6)_{16}$

Conversion du décimal vers l'hexadécimal

Méthode : On divise le nombre par 16 et on note le quotient et le reste. On Répète cette opération jusqu'à ce que le quotient soit 0.

Exemple 1 : 47_{10}→ 47 ÷ 16 = 2 (reste 15, soit F) **Résultat : $47_{10} = 2F_{16}$**

Exemple 2 : 255_{10} →

Etape 1 : 255 ÷ 16 = 15 avec un reste de 255 – (15*16) = 15

Etape 2 : 15 ÷ 16 = 0 avec un reste de 15– (0*16) = 15

En lisant les restes de bas en haut, on obtient $(FF)_{16}$

Exemple 3 : 100_{10} est équivalent à 64_{16}.

Représentation des nombres

Représentation des nombres en mémoire

La représentation des nombres en mémoire, surtout dans le contexte des ordinateurs, implique de savoir comment gérer à la fois les nombres positifs et négatifs. Les principales méthodes utilisées pour cette représentation sont :

1. **Nombres positifs en binaire**

   Les nombres positifs sont représentés directement en binaire. Par exemple, pour le nombre +5 :
   Représentation de +5 en binaire (base 2) :
   5 en décimal se convertit en binaire de la façon suivante :

   $$\mathbf{5 = 4+1 = 2^2 + 2^0 = 101_2}$$

   En mémoire, +5 serait stocké comme 00000101 en utilisant un format de 8 bits (avec des zéros devant pour compléter

   Plage de valeurs :
   Pour n-bits : De 0 à 2^n-1
   Exemple, avec 8 bits, Plage : 0 à 255

Représentation des nombres

2. Nombres négatifs en binaire

Les nombres négatifs ne peuvent pas être directement représentés en binaire. On utilise donc des systèmes comme **le complément à un** et **le complément à deux.**

Plage de valeurs :
Pour n-bits : De -2^{n-1} à $2^{n-1}-1$
Exemple, avec 8 bits, Plage : −128 à 127

A - Complément à un

Le complément à un d'un nombre binaire est obtenu en inversant tous les bits (0 devient 1 et 1 devient 0).

Exemple : **Représentation de -5 en complément à 1 :**

Représentation de +5 → 00000101
Complément à un : 11111010 (inversion de tous les bits)

B - Complément à deux

Le complément à deux est obtenu en prenant le complément à un et en ajoutant 1 au résultat. C'est la méthode la plus courante pour représenter les nombres négatifs en informatique.

Exemple : **Représentation de -5 en complément à 2** :
1 - Complément à un de +5 : 11111010
2- On ajoute **1** : **11111010** + 1 = **11111011** qui est la représentation de **-5** en complément à **2**

Représentation des nombres

2. Nombres à virgule flottante

Les nombres à virgule flottante sont utilisés pour représenter des valeurs réelles (décimales) dans les ordinateurs. Pour cela, les ordinateurs s'appuient sur la norme **IEEE 754**, qui définit la manière de stocker ces nombres en mémoire.

Structure d'un nombre à virgule flottante

Un nombre à virgule flottante est généralement représenté sous la forme suivante :

$$\text{Valeur} = \text{mantisse} \times 2^{\text{exposant}}$$

Mantisse : Représente les chiffres significatifs du nombre. C'est la partie qui contient les données réelles.
Exposant : Indique l'échelle du nombre (c'est-à-dire combien de fois la mantisse doit être multipliée par 2).

Formats de représentation

La norme IEEE 754 définit plusieurs formats pour représenter les nombres à virgule flottante, les deux plus courants sont : **Format simple précision (32 bits)** **et le** **Format double précision (64 bits)**

Représentation des nombres

Format simple précision (32 bits)

Répartition des bits :

- ❑ **1 bit** pour le signe (0 pour positif, 1 pour négatif), **le bit de poids fort**
- ❑ **8 bits** pour l'exposant
- ❑ **23 bits** pour la mantisse **(les bits situés après la virgule)**



Exemple : Représentation du nombre **-5,75**

On code d'abord la valeur absolue en binaire $5{,}75_{10}$ = $101{,}11_2$

Mettons ce nombre sous la forme : **1, *mantisse* : 101,11 → 1,0111.2^2 ; 2^2** décale la virgule de **2** chiffres vers la droite

La Mantisse, étendue sur 23 bits est 0111 0000 0000 0000 0000 000

L'Exposant = 127 + 2 = 129_{10} = $1000\ 0001_2$

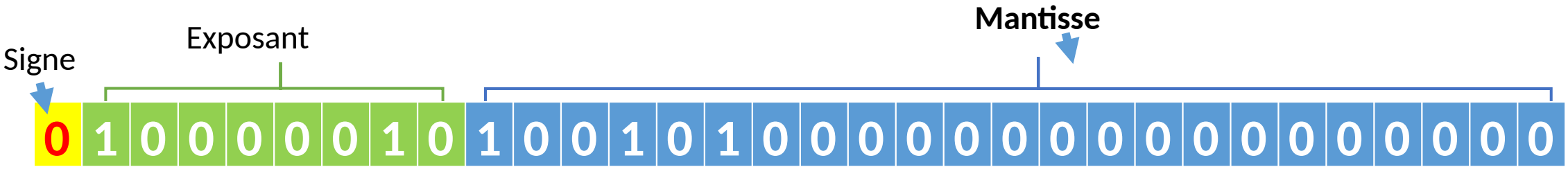
Le Signe = 1, ce qui indique un nombre négatif



Représentation des nombres

Exemple 2 : Représentation du nombre **12,625 = + 1,100101 * 2^3**

Ici l'exposant décalé vaut : 3 + 127 = 130



La représentation IEE754 du nombre **12,625 vaut : 0100 0001 0100 1010 0000 0000 0000 0000**

$(414A0000)_{16}$

0,375*2=0,75 → 0 R 0,75
0,75*2=1,5 → 1 R 0,5
0,5*2=1 → 1 R 0

Exemple 3 : Représentation du nombre **0,375 = (0,011)2 = 1,1 * 2^{-2}**

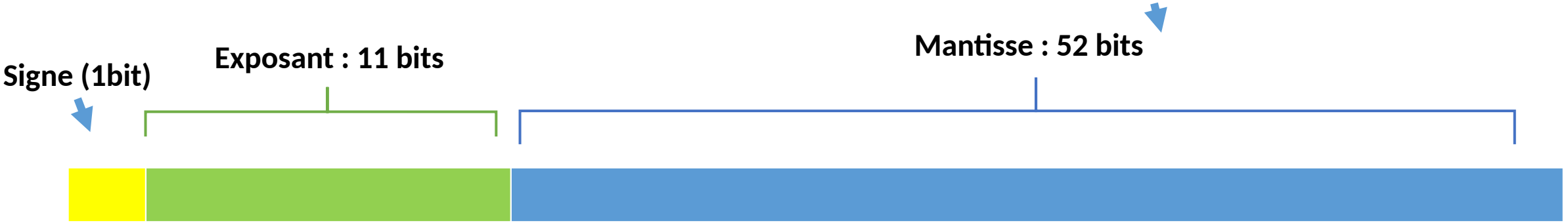
Le nombre est + donc le bit de signe vaut 0
L'exposant vaut -2 et l'exposant décalé : -2+127=125 qui donne en binaire 0111 1101
La mantisse vaut : 1000 0000 0000 0000 0000 000

Résultat : La représentation du nombre **0,375** est : 0011 1110 1100 0000 0000 0000 0000 0000 = **3EC00000**

Question : Que vaut -0,375 ?

Représentation des nombres

Format Double précision (64 bits)



La représentation IEE754 du nombre **12,625 vaut :**

$12,625 = 1100,101 = 1,100101.2^{*3}$

Le nombre est positif donc le bit de signe vaut 0

L'exposant vaut +3 et l'exposant décalé donne maintenant : +3+1023=1026 qui donne en binaire 1000 0000 010

La mantisse vaut : 1001 0100 0000 0000 0000

Résultat : La représentation du nombre **12,625** est :

0100 0000 0010 1001 0100 0000 0000 . . . . . .0000 qui donne $(4029400000000000)_{16}$

Question : Que vaut -12,625 ?

Représentation des nombres

Nombres limites : +0, −0, +∞, et −∞ ont des représentations spéciales en IEEE754 (simple précision)

+0 :
Signe : 0
Exposant : 00000000 (tout à 0)
Mantisse : 00000000000000000000000 (tout à 0)

-0 :
Signe :1
Exposant : 00000000 (tout à 0)
Mantisse : 00000000000000000000000 (tout à 0)

+∞ (Infini positif)
Signe : 0
Exposant : 11111111 (tout à 1)
Mantisse : 00000000000000000000000 (tout à 0)

-∞ (Infini négatif)
Signe : 1
Exposant : 11111111 (tout à 1)
Mantisse : 00000000000000000000000 (tout à 0)

NaN **(Not a Number)**
Signe : 0 ou 1 (peu importe)
Exposant : 11111111 (tout à 1)
Mantisse : Non nulle (n'importe quelle valeur sauf tout 0)

Représentation du texte en binaire (ASCII)

Nous avons vu précédemment que **l'ordinateur ne comprend que des mots binaires**, c'est-à-dire des séquences de **0 et de 1**. Nous avons déjà vu comment représenter des **nombres entiers** ou **réels** à l'aide de normes comme **IEEE 754**. Cependant, l'ordinateur doit aussi **manipuler des caractères** tels que des **lettres, chiffres ou symboles**. Ces caractères doivent eux aussi être **codés en binaire** pour être interprétés par la machine.
Dans ce contexte, plusieurs standards ont été proposés pour **représenter ces caractères de manière uniforme**, et parmi eux, **ASCII** (American Standard Code for Information Interchange) est l'un des plus célèbres.

Norme ASCII

Le standard ASCII, développé dans les années 1960, est toujours fondamental pour comprendre comment les caractères sont encodés en binaire. Cependant, son **usage est aujourd'hui limité** en raison des **restrictions du nombre de caractères** qu'il peut représenter, ce qui a conduit à l'adoption de normes plus universelles comme **UNICODE**.

Le code ASCII est **normalisé par l'ANSI** (American National Standards Institute), qui s'occupe des normes aux États-Unis pour les produits, systèmes et services.

Représentation du texte en binaire (ASCII)

Caractéristiques du Code ASCII

ASCII de Base (7 bits)

Encodage sur 7 bits : 128 caractères possibles
Caractères inclus :

Lettres **majuscules** et **minuscules** de l'alphabet anglais
Chiffres (0 à 9)
Symboles de ponctuation et autres caractères de base comme $ ou @
Caractères de contrôle : Ils permettent d'ordonner des actions comme le saut de ligne ou la fin de texte.

Limites du Code ASCII

ASCII a été conçu pour la langue anglaise, ce qui signifie qu'il ne prend pas en charge les caractères accentués ou les lettres spécifiques à d'autres langues.

ASCII Étendu (8 bits)

Pour remédier aux limitations du code ASCII de base, une version étendue sur 8 bits a été introduite. Elle permet d'encoder 256 caractères et inclut :Lettres accentuées (**é, à, ç,** etc.) ainsi que des symboles supplémentaires.

Représentation du texte en binaire (ASCII)

Exemples de Codage ASCII

Caractères Spéciaux et Contrôles

Caractères non affichables (codes 0 à 31) : utilisés pour des commandes de contrôle (saut de ligne, fin de texte, etc.).
Espace : code 32.
Point d'exclamation : code 33.
Chiffres : de 48 à 57 (ex. : 48 pour 0, 49 pour 1).
Majuscules : de 65 (A) à 90 (Z).
Minuscules : de 97 (a) à 122 (z).

Pour passer du code d'une lettre majuscule à celui de la même lettre en minuscule, on ajoute **32** :
Exemple : **B** a pour code **66**. En ajoutant **32**, on obtient **98**, qui correspond à b.

Limites du Code ASCII

ASCII a été conçu pour la langue anglaise, ce qui signifie qu'il ne prend pas en charge les caractères accentués ou les lettres spécifiques à d'autres langues.

ASCII Étendu (8 bits)

Pour remédier aux limitations du code ASCII de base, une version étendue sur 8 bits a été introduite. Elle permet d'encoder 256 caractères et inclut :Lettres accentuées (**é, à, ç,** etc.) ainsi que des symboles supplémentaires.

Représentation du texte en binaire (ASCII)

Exemples de Codage ASCII

Exemple : utilisons l'ASCII étendu pour représenter le mot « Bonjour »

B : 01000010

o : 01101111

n : 01101110

j : 01101010

o : 01101111

$b_7 b_6 b_5$ → / Bits	000	001	010	011	100	101	110	111
b_4 b_3 b_2 b_1 / Column → / Row ↓	0	1	2	3	4	5	6	7
0 0 0 0	0	NUL	DLE	SP	0	@	P	`	p
0 0 0 1	1	SOH	DC1	!	1	A	Q	a	q
0 0 1 0	2	STX	DC2	"	2	B	R	b	r
0 0 1 1	3	ETX	DC3	#	3	C	S	c	s
0 1 0 0	4	EOT	DC4	$	4	D	T	d	t
0 1 0 1	5	ENQ	NAK	%	5	E	U	e	u
0 1 1 0	6	ACK	SYN	&	6	F	V	f	v
0 1 1 1	7	BEL	ETB	'	7	G	W	g	w
1 0 0 0	8	BS	CAN	(	8	H	X	h	x
1 0 0 1	9	HT	EM	)	9	I	Y	i	y
1 0 1 0	10	LF	SUB	*	:	J	Z	j	z
1 0 1 1	11	VT	ESC	+	;	K	[	k	{
1 1 0 0	12	FF	FS	,	<	L	\	l	|
1 1 0 1	13	CR	GS	—	=	M	]	m	}
1 1 1 0	14	SO	RS	.	>	N	^	n	~
1 1 1 1	15	SI	US	/	?	O	_	o	DEL

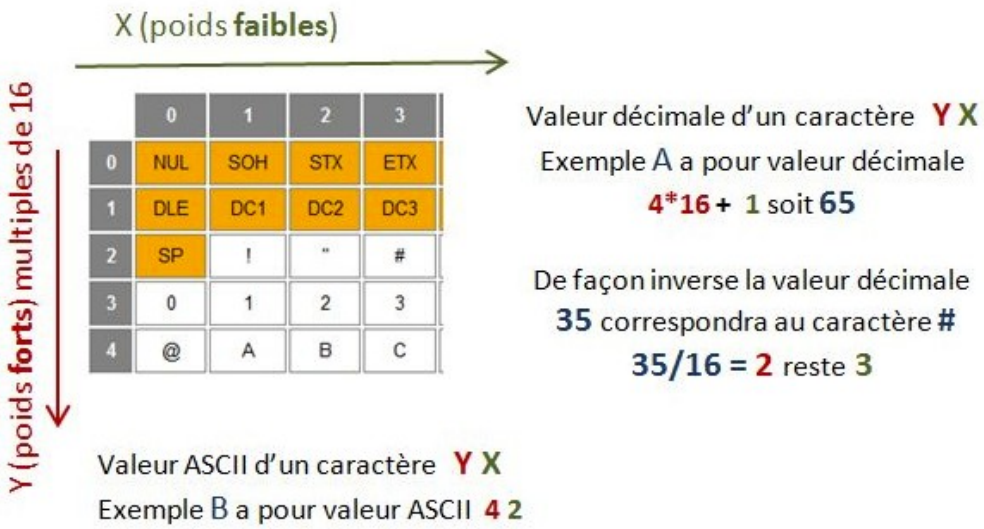


Table des codes ASCII étendu

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	TAB	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2	SP	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	
8	€	□	‚	ƒ	„	…	†	‡	ˆ	‰	Š	‹	Œ	□	Ž	□
9	□	‘	’	“	”	•	–	—	˜	™	š	›	œ	□	ž	Ÿ
A		¡	¢	£	¤	¥	¦	§	¨	©	ª	«	¬		®	¯
B	°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
C	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
D	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
E	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
F	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

Représentation du texte en binaire (ASCII)

Malgré son utilité historique, ASCII ne constitue plus la norme de base dans les systèmes modernes, principalement à cause de :

- ❑ La limitation à 256 caractères (même en version étendue).
- ❑ L'incapacité de représenter des lettres non latines (comme l'arabe, le grec ou le cyrillique).
- ❑ L'impossibilité de prendre en charge les **emojis** ou d'autres symboles modernes.

Pour surmonter les limites d'ASCII, la norme UNICODE a été développée en 1991. Elle permet de représenter :

- ❑ Tous les systèmes d'écriture existants.
- ❑ Les emojis et autres symboles modernes.

Unicode est uniquement une table qui regroupe tous les caractères existant au monde, il ne s'occupe pas de la façon dont les caractères sont codés dans la machine. Unicode accepte plusieurs systèmes de codage : UTF-8, UTF-16, UTF-32. Le plus utilisé, notamment sur le Web, est UTF-8.

Pour encoder les caractères Unicode, UTF-8 utilise un nombre variable d'octets : les caractères "classiques" (les plus couramment utilisés) sont codés sur un octet, alors que des caractères "moins classiques" sont codés sur un nombre d'octets plus important (jusqu'à 4 octets). Un des avantages d'UTF-8 c'est qu'il est totalement compatible avec la norme ASCII : Les caractères Unicode codés avec UTF-8 ont exactement le même code que les mêmes caractères en ASCII.

Quelques caractères spécifique de contrôle

Caractères spécifiques de contrôle

Nom	*Commande*	*Action*
Commandes de format		
CR	Carriage return	Retour chariot : retour en début de ligne
LF	Line feed	Avancer d'une ligne : passage à la ligne suivante
BS	Backspace	Espace arrière : suppression du caractère précédent
HT	Horizontal tabulation	Tabulation horizontale : déplacement dans la ligne pour aligner le texte qui suit
VT	Vertical tabulation	Tabulation verticale
SP	Space	Espace
FF	Form feed	Avancer d'une feuille : passer à la page suivante
DEL	Delete	Supprime le caractère qui suit
BEL	Bell	Émet un « bip » ou un autre avertissement sonore

Opérations binaires

Les opérations binaires suivent des règles similaires aux opérations décimales, avec certaines spécificités.

A- Addition Binaire

Les règles de l'addition binaire sont simples :

0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 10 (**ce qui équivaut à 0 avec une retenue de 1**)

Exemple :
1011 + 1101 = 11000 = 24(Base 10)
1110 + 101 = 10011 = 19(Base 10)
110101,1101 + 11,011 = 111001,0011
100011,11 + 101,101 = 101001,011

B- Soustraction Binaire

La soustraction suit les règles :

0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1 (**nécessite un emprunt de la position suivante**)

Exemple :
10101 - 1101 = 1000 = 8(Base 10)
10010 - 1101 = 101 = 5(Base 10)
101101,11011 – 11101,111 = 1111,11111

Opérations binaires

Les opérations binaires suivent des règles similaires aux opérations décimales, avec certaines spécificités.

C- Multiplication Binaire

La multiplication en binaire est plus directe, avec les règles :

0 × 0 = 0
0 × 1 = 0
1 × 0 = 0
1 × 1 = 1

Exemple 1

```
   110
 × 101
---------
   110    (110 × 1)
+  000    (110 × 0, décalé d'un cran)
+ 110     (110 × 1, décalé de deux crans)
---------
 11110
```

110 = 6(Base 10)
101 = 5(Base 10)
6 × 5 = 30(Base 10) = 11110(Base2)

Exemple 2

```
   111
 ×  11
---------
   111   (111 × 1)
+  111   (111 × 1, décalé d'un cran)
---------
 10101
```

111 = 7(Base 10)
11 = 3(Base 10)
7 × 3 = 21(Base 10) = 10101(Base2)

Exemple 3: 111000011,11101*110,1 = 101101111001,011001



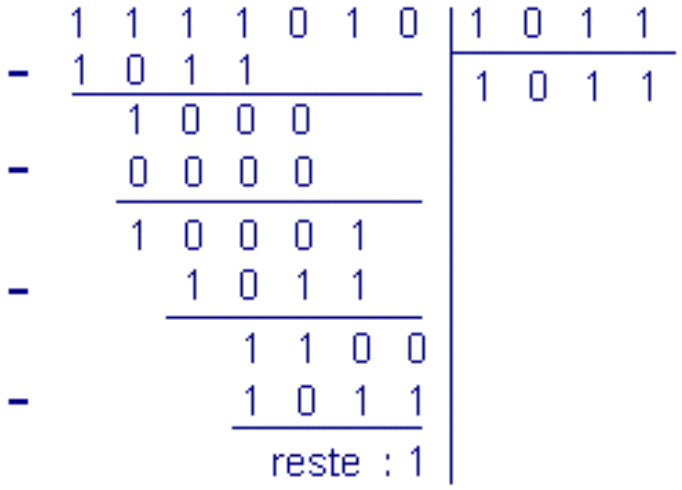
Opérations binaires

Les opérations binaires suivent des règles similaires aux opérations décimales, avec certaines spécificités.

D- Division binaire

La division binaire est similaire à la division décimale, mais elle utilise uniquement les chiffres 0 et 1

Exemple 1



1111010 = 122(Base 10)
1011 = 11(Base 10)
122 ÷ 11 = 11.090909090909092(Base 10) =
1011.0001011110100010111(Base 2)

Exemple 2 : 1000 1110 000 / 111 = 10100010 + Reste (10)

1 : Introduction au langage C

I.1 Qu'est-ce que le langage C ?

Le langage C est l'un des langages de programmation les plus importants et influents dans l'histoire de l'informatique. Développé dans les années 1970 par **Dennis Ritchie** chez **AT&T Bell Labs**, il a été conçu pour la création du système d'exploitation **Unix**. Depuis, il a été utilisé pour écrire de nombreux systèmes d'exploitation, ainsi que des applications nécessitant des performances optimales et un contrôle précis sur le matériel.

C'est un langage de programmation structuré, mais très "près" de la machine.
Publication en 1978 de "The C programming language" par Kernighan et Ritchie: définition classique du C.
Le développement de compilateurs C par d'autres maisons ont rendu nécessaire la définition d'un standard précis: le standard ANSI-C.
1983: Développement par AT&T du C++
1988: Seconde édition du livre "The C programming language"
1990: Standard ANSI-C++

Le langage C est très utilisé dans le développement de logiciels systèmes tels que les systèmes d'exploitation (comme Linux), les compilers, les applications embarquées, les applications en temps réel, et bien d'autres domaines où la performance et la gestion de la mémoire sont cruciales.

I.2 Structure d'un programme C

Un programme en C est composé de plusieurs éléments de base qui doivent être organisés de manière spécifique pour être exécutés correctement par un compilateur. Voici la structure générale d'un programme en C :

```c
#include <stdio.h> // Bibliothèque standard

// Déclaration de la fonction main() où l'exécution commence
int main() {

    // Code exécuté par le programme
    printf("Hello, World!\n"); // Affiche un message à l'écran

    return 0; // Retourne un statut de réussite
}
```

```c
#include <stdio.h> // Bibliothèque standard

// Déclaration de la fonction main() où l'exécution commence
int main() {

    // Code exécuté par le programme
    printf("Hello, World!\n"); // Affiche un message à l'écran

    return 0; // Retourne un statut de réussite
}
```

#include <stdio.h> : directive de préprocesseur. Elle inclut une bibliothèque standard (ici, stdio.h qui est la bibliothèque pour les entrées/sorties)

int main() La fonction main est le point d'entrée du programme. C'est ici que commence l'exécution du programme. Le type **int** indique que la fonction retourne un entier (généralement pour signaler le succès ou l'échec de l'exécution).

printf("Hello, World!**\n**"); : Cette ligne appelle la fonction **printf(),** qui est utilisée pour afficher un message à l'écran. Le **\n** indique un saut de ligne

return 0; : La fonction **main()** retourne un entier. Le **0** renvoie un code de succès au système d'exploitation

1.3 Le processus de compilation

Le langage C est un langage **compilé**, ce qui signifie que le code source est transformé en un **fichier exécutable binaire** que l'ordinateur peut exécuter directement. La compilation se déroule en plusieurs étapes :

Préprocesseur : Cette étape traite les directives de préprocesseur, comme **#include**, et génère un fichier source modifié prêt à être compilé.

Compilation : Le compilateur traduit le code source en code objet **(fichier .o)**, qui est une version binaire partielle du programme.

Édition des liens : Cette étape lie les fichiers objets avec les bibliothèques nécessaires pour créer le fichier exécutable final.

Exemple de compilation sur un système Unix ou Linux

```
gcc -o mon_programme mon_programme.c
```



gcc : (GNU Compiler Collection)
GNU n'est pas **Linux**.

GNU fournit les outils
Linux fournit le noyau (kernel)

```
GCC : compilateur C/C++
GDB : débogueur
Make : automatisation de compilation
glibc : bibliothèque standard C
binutils : assembleur, éditeur de liens (ld)
bash : interpréteur de commandes
```

Cette commande compile le fichier **mon_programme.c** et produit un exécutable nommé mon_programme.

Cela génère un fichier exécutable nommé **mon_programme**

On peut l'exécuter en tapant sur le bash : **./mon_programme**

1.3 – A- Configurer l'environnement

Exemple avec Visual Studio Code (VSCode)

1- Installation de Visual Studio Code

Téléchargez puis Installer Visual Studio Code en se rendant sur le site officiel de VSCode : https://code.visualstudio.com

2- Installation d'un compilateur

Installer un compilateur C car VSCode n'inclut pas de compilateur C. Le plus couramment utilisé sur Windows est MinGW (Minimalist GNU for Windows) à cette adresse : https://sourceforge.net/projects/mingw/files/Installer/

3- Ajouter MinGW au PATH :

Allez dans Paramètres **Système -> Avancé -> Variables d'environnement**. Sous Variables système, recherchez la variable Path et cliquez sur Modifier. Ajoutez le chemin du dossier bin de **MinGW** à la liste. Cela ressemblera à quelque chose comme **C:\mingw\bin**.

4- Vérifier l'installation de MinGW :

Ouvrez une fenêtre Invite de commandes (Cmd) et tapez gcc --version pour vérifier que MinGW est installé correctement.

1.3 – A- Configurer l’environnement

6- Installer l'extension Code Runner

Code Runner est une extension populaire pour Visual Studio Code qui permet d'exécuter facilement des programmes dans plusieurs langages, dont C, directement depuis l'éditeur,.

A- Ouvrez Visual Studio Code.

B- Allez dans le Marketplace des extensions en cliquant sur l'icône Extensions

C- Recherchez "Code Runner" dans la barre de recherche et cliquez sur Installer à côté de l'extension Code Runner

Sur Internet, Vous pouvez aussi tester votre programme en C à cette adresse :

https://www.onlinegdb.com/online_c_compiler

1.4 Les commentaires en C

Les commentaires dans le code ne sont pas exécutés. Ils servent à expliquer ou à documenter le code

En C, il existe deux types de commentaires :

A- Commentaire sur une ligne : Utilisé pour ajouter des commentaires courts.

```
// Ceci est un commentaire sur une seule ligne
```

B- Commentaire multi-lignes : Utilisé pour commenter plusieurs lignes.

```
/* Ceci est un commentaire
qui peut s'étendre sur plusieurs lignes.
*/
```

Exemple d'erreur:

```
/* ceci est un commentaire correct */
/* ceci est /* évidemment */ incorrect */
```

I. 5 Premier programme en C

Le classique "bonjour" (affiche bonjour à l'écran). Pour le faire, il faut:

- Inclure les bibliothèques
- Inclure le main

```
main()
{
déclaration des variables:  ici aucune
instruction: écrire "bonjour"
}
```

La fonction prédéfinie qui permet d'écrire à l'écran est **printf**, elle est contenue dans le fichier en-tête **stdio.h**; sa syntaxe est: **printf("ce que l'on veut écrire");**

Notre premier programme:

```
#include <stdio.h>
main()
{
printf("bonjour\n"); /*toute instruction se termine par un point
virgule*/
}
```

I. 6 les séquences d'échappement

```
printf("bonjour\n")
```

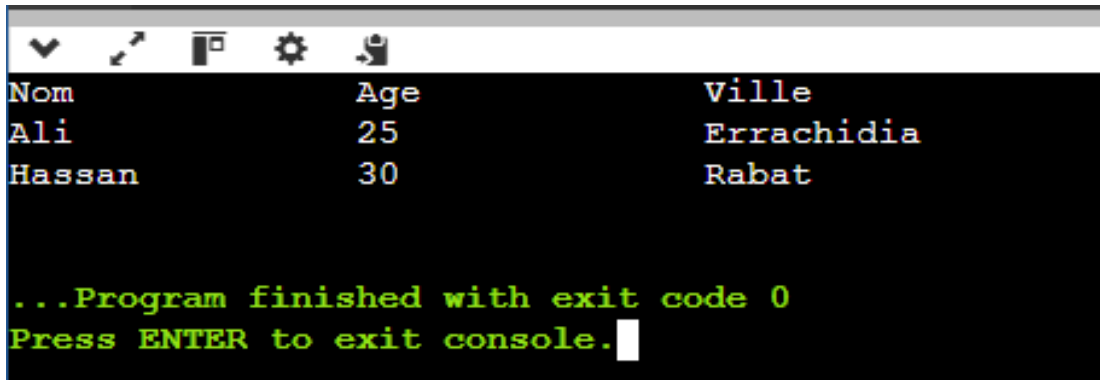
La suite de symboles \n (antislash n) à la fin de la chaîne de caractère est la notation **C** signifiant le passage à la ligne (**n comme new ligne)**

Le langage C dispose de plusieurs couples de symboles qui contrôlent l'affichage du texte.

\t tabulation
\n nouvelle ligne
\b batch (curseur arrière)
\r return (retour au début de ligne, sans saut de ligne)
\a attention (signal acoustique)

Exemple de code avec \t (tabulation)

```
#include <stdio.h>
int main() {
   // Affichage d'un tableau de nombres avec des tabulations pour un alignement
propre
   printf("Nom\t\tAge\t\tVille\n");
   printf("Ali\t\t25\t\tErrachidia\n");
   printf("Hassan\t\t30\t\tRabat\n");
   return 0;
}
```



I. 6 les séquences d'échappement (Exemple)

Nouvelle ligne et retour chariot

```
#include <stdio.h>
int main() {
    printf("Bonjour\rSalut\n");
    return 0;
}
```

Résultat : Salutur

`\r` ramène le curseur au début de la ligne
`"Salut"` écrase `"Bonjour"`

Retour arrière \b

Quel est le résultat de ce programme ?

```
#include <stdio.h>
int main() {
    printf("BONJOUR\b\b\b\bsoir");
    return 0;
}
```

Résultat : BONsoir

`\b` supprime les trois dernières lettres **(OUR)**

`Elles sont remplacées par "soir"`

Signal sonore

```
#include <stdio.h>
int main() {
    printf("Attention\a\n");
    return 0;
}
```

Résultat : Attention et émet un bip sonore (selon le système)

1.7 Types de base, opérateurs, expressions

Dans un programme, on trouve des **variables** et des **constantes**, Il faut fixer leurs types. Pour produire de nouvelles valeurs, les variables et les constantes peuvent être combinées à l'aide **d'opérateurs** dans des **expressions,**

A) Les types de base

En mathématiques, on distingue divers ensembles de nombres (entiers naturels (ℕ), entiers relatifs (Z), réels (ℝ), complexes (ℂ),...). L'ordre de grandeur des nombres est illimité, ils peuvent être exprimés sans perte de précision.

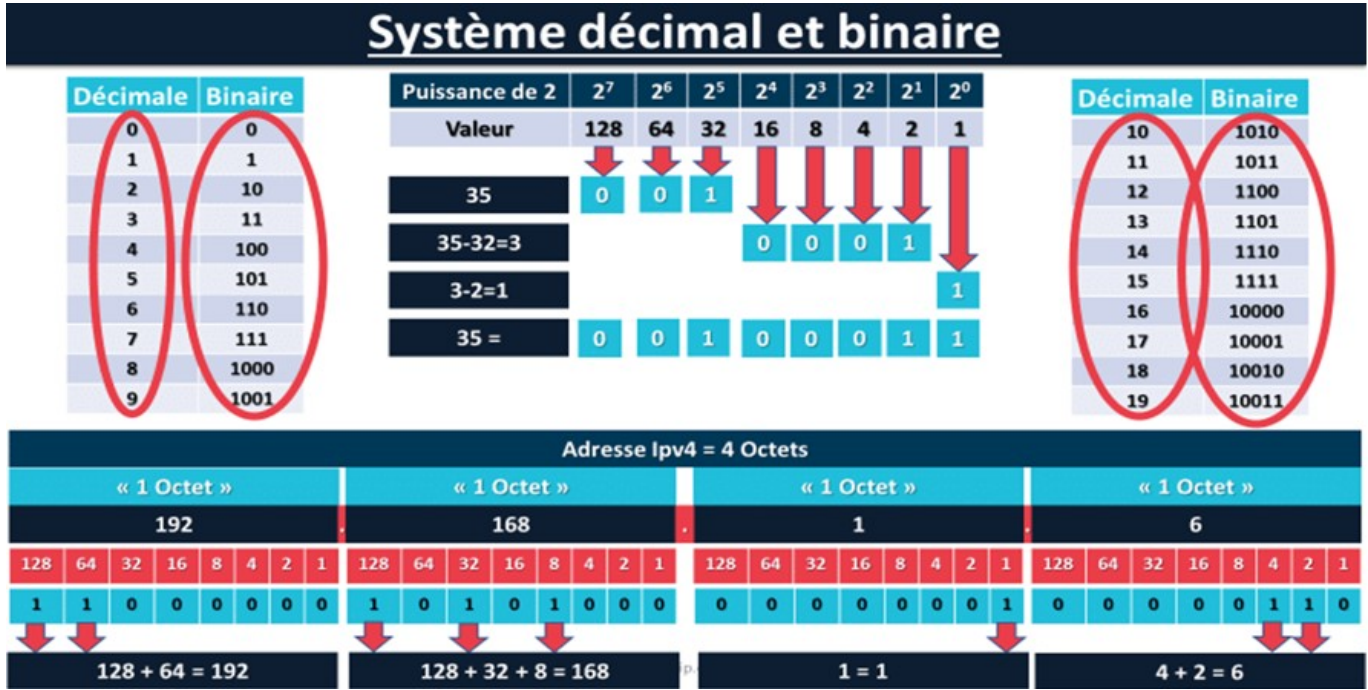
ℕ= {0;1;2;3;4... }.

Z = {...−3;−2;−1;0;1;2;3...}.

ℝ : 2, 0, -5, 0.67, 1 3 , 3

Un ordinateur utilise le système binaire pour sauvegarder et calculer les nombres, il existe pour un ordinateur deux grands systèmes de nombres: les **entiers** et les **rationnels**.

Rappel sur
Les systèmes de numération



1.7 Types de base, opérateurs, expressions

- Les entiers

Définition	Description	Valeur min	Valeur max	Nombre d'octets
char	caractère	-128	127	1
short	entier court	-32768	32767	2
int	entier standard	-32768	32767	2
long	Entier long	-2147483648	2147483647	4

Si l'on ajoute le préfixe **unsigned** (non signé), les domaines sont déplacés ainsi:

Définition	Valeur min	Valeur max
unsigned char	0	255
unsigned short	0	65535
unsigned int	0	65535
unsigned long	0	4294967295

Les valeurs des limites des différents types sont indiquées dans le fichier **limits.h**.

1.7 Types de base, opérateurs, expressions

- Les rationnels

Définition	Précision	Mantisse*	Valeur min	Valeur max	Nb d'octets
float	simple	6	$3{,}4.10^{-38}$	$3{,}4.10^{+38}$	4
double	double	15	$1{,}7.10^{-308}$	$1{,}7.10^{+308}$	8
long double	avancée	19	$3{,}4.10^{-4932}$	$1{,}1.10^{+4932}$	10

* La mantisse est le nombre de chiffres significatifs après la virgule.

- Les variables booléennes

En langage C, Il n'existe pas de type spécifique pour les variables booléennes, tous les types de variables numériques peuvent être utilisés pour exprimer des opérations logiques:

La variable logique **FAUX** correspond à **la valeur numérique 0**.
La variable logique **VRAI** correspond **toute valeur différente de 0**.

1.8 - Déclaration de variables

En langage C, la déclaration des variables permet de définir **un nom** pour une variable et de lui associer un **type**.
Cela permet au compilateur de savoir quel genre de données la variable va stocker (comme un **entier**, un **flottant**, un **caractère**, etc.).

Syntaxe de base :

type nom; // Déclaration d'une variable de type '**type**' avec le nom '**nom**'.

Exemple

```
int a;  // Déclare une variable entière appelée 'a'

int a, b, c;  // Déclare trois variables entières : a, b, et c

int a = 1, b = 2, c = 3;  // Déclare et initialise 'a', 'b' et 'c' respectivement à 1, 2 et 3

float pi = 3.14;  // Déclare une variable 'pi' de type 'float' et l'initialise avec la valeur 3.14
```

2- Les opérateurs standards

- L'affectation (=)

L'opérateur d'affectation en C est représenté par le signe =.Il permet d'assigner une valeur à une variable.

Syntaxe : **Nom_variable = expression**;

Affectation avec des Valeurs Constantes

```
Lettre = 'L'; // Affectation d'une constante à Lettre
Pi = 3.1416; // Affectation d'une constante à Pi
```

Affectation avec des Valeurs Variables

```
a = b;          // La valeur de b est affectée à a
```

Affectation avec des Expressions

```
x = a + b;      // Affectation du résultat de l'expression a + b à x
```

Exemple

```
i = i + 2;
En C, on peut écrire:
i+ = 2;
```

Cette formulation (i+ = 2) suit la logique humaine: on ajoute 2 à i.

2- Les opérateurs standards

- Les opérateurs arithmétiques

Il permettent de réaliser des opérations mathématiques de base sur des variables ou des constantes.

+ addition
- soustraction
*** multiplication**
/ division
% modulo (reste de la division entière) (par exemple, 5%2=1)

- Les opérateurs logiques

&& ET	a && b; // Vrai si a et b sont vrais
|| OU	a || b; // Vrai si a ou b (ou les deux) sont vrais
! NON	!a; // Vrai si a est faux, et faux si a est vrai

Exemple :
32&&2.3 → 1
!65,43 → 0
0||!(32>12) → 0

Les résultats des opérations logiques sont de type **int**: la valeur **0** correspond à la valeur booléenne **FAUX**, la valeur **1** correspond à la valeur booléenne **VRAI**.

2- Les opérateurs standards

- Les opérateurs de comparaison

Les opérateurs de comparaison en C sont utilisés pour comparer deux valeurs. Ils retournent un résultat booléen (vrai ou faux), souvent utilisé dans les instructions conditionnelles.

== EGALITE **!= INEGALITE**

< INFERIEUR **<= INFERIEUR OU EGAL**

> SUPERIEUR **>= SUPERIEUR OU EGAL**

Exemple d'utilisation

```
int a = 10, b = 5;
int result1 = a == b;  // result1 = 0 (faux, car a n'est pas égal à b)
int result2 = a != b;  // result2 = 1 (vrai, car a est différent de b)
int result3 = a > b;   // result3 = 1 (vrai, car a est supérieur à b)
int result4 = a < b;   // result4 = 0 (faux, car a n'est pas inférieur à b)
```

2- Les opérateurs standards

- L'opérateur d'incrémentation et de décrémentation

Les opérateurs d'incrémentation **(++)** et de décrémentation **(--)** en C permettent de modifier la valeur d'une variable en l'augmentant ou en la diminuant de **1**. Ces opérateurs sont très couramment utilisés dans des boucles ou pour des manipulations simples de variables.

i = i + 1 s'écrit: i++ ou ++i

i = i – 1 s'écrit: i-- ou --i

Remarque

Pour incrémenter ou décrémenter une variable, par exemple dans une boucle (dans ce cas pas de différence entre la notation **préfixe (++i,--i)** et la notation **postfixe (i++,i--).**

Pour incrémenter ou décrémenter une variable et en même temps affecter sa valeur à une autre variable.

Dans ce cas, on choisit entre la notation **préfixe** et **postfixe**:

2- Les opérateurs standards

Exemple

X = i++; passe d'abord la valeur de i à X, puis incrémente i (le ++ est après i, on l'incrémente après)

X = i--; passe d'abord la valeur de i à X, puis décrémente i

X = ++i; incrémente d'abord i puis passe la valeur de i incrémentée à X (le ++ est avant i, on l'incrémente avant)

X = --i; décrémente d'abord i puis passe la valeur de i décrémentée à X

Application

N = 5;

X = N++; Résultat: **X = 5 et N = 6**

N = 5;

X = **++N**; Résultat: **X = 6 et N = 6**

2- Les opérateurs standards

- Priorités des opérateurs

Priorité 1 (la plus élevée)	( ) : Parenthèses	
Priorité 2	!, ++, --	Opérateurs unaires
Priorité 3	*, /, %	Opérateurs arithmétiques :
Priorité 4	+, -	
Priorité 5	<, <=, >, >=	Opérateurs de comparaison :
Priorité 6	==, !=	
Priorité 7	&&	Opérateurs logiques :
Priorité 8	||	
Priorité 9 (la plus basse)	=, +=, -=, *=, /=, %=	Opérateurs d'affectation :

2- Les opérateurs standards

- Priorités des opérateurs

Priorité 1 (la plus élevée)	( ) : Parenthèses	
Priorité 2	!, ++, --	Opérateurs unaires
Priorité 3	*, /, %	Opérateurs arithmétiques :
Priorité 4	+, -	
Priorité 5	<, <=, >, >=	Opérateurs de comparaison :
Priorité 6	==, !=	
Priorité 7	&&	Opérateurs logiques :
Priorité 8	||	
Priorité 9 (la plus basse)	=, +=, -=, *=, /=, %=	Opérateurs d'affectation :

2- Les opérateurs standards

- Priorités des opérateurs (Cas de Priorités Égales)

Dans ce cas, l'ordre d'évaluation des opérateurs est déterminé par l'associativité. Il existe deux types d'associativité : **associativité de gauche à droite** et **associativité de droite à gauche**.

Les opérateurs de même priorité sont évalués selon des règles spécifiques :

- Les opérateurs binaires (comme +, -, *, &&, etc.) de la même classe ont une associativité de gauche à droite.
- Les opérateurs unaires (!, ++, --) et les opérateurs d'affectation (comme =, +=, -=, etc.) ont une associativité de droite à gauche.

Exemple 1 :
10 + 20 + 30 – 40 sera évaluée comme suit:
10 + 20) → 30
30 + 30 → 60
60 – 40 → 20

Exemple 2 :
pour A = 3 et B = 4
A* = B+ = 5 sera évaluée comme suit:
B+ = 5 → B = 9
A* = 9→ A = 27

3- Conversions de type

- Calculs et affectations

Si un opérateur a des opérandes de types différents, les valeurs des opérandes sont converties automatiquement dans un type commun.
Cette conversion implicite s'effectue en général des types "plus petits" vers les types "plus larges", afin de ne pas perdre en précision.

Remarque

Lors d'une affectation, la donnée à droite du signe "=" est convertie dans le type à gauche du signe "=".
Dans ce cas, il peut y avoir perte de précision.

Exemple :

```
int a = 5;
double b = 12.34;
float c;
c = a + b;
```

Lors de l'opération **a + b**, la variable **a** de **type int** est convertie en double pour pouvoir être additionnée avec **b** (qui est de **type double**). Le résultat de l'addition est ensuite affecté à **c**, une variable de **type float**. Cette conversion de double à **float** peut entraîner une perte de précision, car le type **float** est moins précis que **double**.

3- Conversions de type

- Appels de fonctions

Lors de l'appel d'une fonction, les paramètres sont automatiquement convertis dans les types déclarés dans la définition de la fonction.

Exemple :

int A =8, Resultat;
Resultat = pow(A, 2);
Appel de pow(): 2 et A sont convertis en doubles, le résultat est double, il est converti en int pour être affecté à Resultat.

Erreur classique:

int A = 5, B = 2;
float C;
C = A / B;
A / B est entier (A et B sont int), donc A / B = 2, converti en float, C = 2.0

```c
#include <stdio.h>

int main()
{
    int A = 5, B = 2;
float C;
C = A / B;
printf("la variable C est %f\n",C);

    return 0;
}
```

Pour avoir un résultat « correct », il faut s'assurer qu'au moins un des opérandes soit un float ou un double avant la division. Exemple : C = (float)A / B

```
la variable C est 2.000000


...Program finished with exit code 0
Press ENTER to exit console.
```

3- Conversions de type

Règles de conversion lors d'une opération :

Opération avec deux entiers (int, short, char, etc.) :

- **Types char et short convertis en int** :
  Lorsqu'une opération implique des types char ou short avec un type int, ces types sont convertis en int avant l'opération, car int est considéré comme un type plus large.
- **Choix du type de résultat** :
  Si un opérande est de type unsigned int et l'autre est de type int :

  Si la plage de int peut contenir toutes les valeurs possibles de l'unsigned int, les deux seront convertis en int.

  Sinon, les deux seront convertis en unsigned int.
- L'échelle des types en C pour les entiers est généralement la suivante :
  char → short → int → unsigned int → long → unsigned long.

Opération avec un entier et un rationnel (float, double, long double) :

- **Conversion de l'entier en type rationnel** :
  une opération entre un entier (int, char, short, etc.) et un nombre à virgule flottante (float, double, ou long double), l'entier est converti en type flottant (float, double, ou long double), en fonction du type du rationnel. Cela garantit que l'opération soit effectuée avec la précision des nombres à virgule flottante.

Exemple

```
int a = 5; float b = 2.5;
float c;
c = a + b;  // a est converti en float, donc l'addition donne un résultat float
```

3- Conversions de type

Règles de conversion lors d'une opération :

Opération avec deux rationnels (float, double, long double) :

Lorsque deux nombres à virgule flottante de types différents sont impliqués dans une opération, l'ordinateur choisit le type le plus large pour éviter la perte de précision.

L'échelle des types flottants est généralement la suivante : float → double → long double.

Exemple

```
float a = 3.5;
double b = 4.5;
double c;
c = a + b; // a est converti en double pour l'opération, donc le résultat est de type double
```

Conversion de type forcée

Il est possible de convertir explicitement une valeur en un type quelconque: (type) expression

Exemple:

int A = 4, B = 3; float C;

C = **(float)**A / B;

Le contenu de A reste inchangé, seule la valeur utilisée dans le calcul est convertie.

4- Lire et écrire des données

La bibliothèque standard **<stdio.h>** contient un ensemble de fonctions qui assurent la communication de la machine avec le monde extérieur.

Les fonctions les plus importantes sont :

Pour la lecture :

- **scanf()** : lecture formatée de données
- **getchar()** : lecture d'un caractère

Pour l'écriture :

- **printf()** : écriture formatée de données
- **putchar()** : écriture d'un caractère

4- Lire et écrire des données

1- Ecriture formatée de données

La fonction **printf()** permet d'afficher du texte, des valeurs de variables ou des résultats d'expressions sur l'écran (le fichier de sortie standard, **stdout**)

Syntaxe :

printf("format", expr_1, expr_2)

"format" est une chaîne de caractère qui peut contenir:
- du texte
- des séquences d'échappement
- des spécificateurs de format (un spécificateur pour chaque expression à afficher)

Exemple

```
#include <stdio.h>
int main() {
    int age = 25;
    float taille = 1.75;
        // Affichage d'un texte avec des variables formatées
    printf("L'âge est %d ans et la taille est %.2f mètres.\n", age, taille);
        return 0;
}
```

4- Lire et écrire des données

Les spécificateurs de format

Les spécificateurs de format commencent toujours par le symbole % et sont utilisés dans la fonction printf() pour indiquer comment afficher les valeurs des variables ou expressions.

Symbole	Type	Description
%d ou %i	Entier relatif (int)	Affiche un entier signé.
%u	Entier naturel (non signé) (int)	Affiche un entier non signé.
%o	Entier exprimé en octal (int)	Affiche un entier en base octale.
%x	Entier exprimé en hexadécimal (int)	Affiche un entier en base hexadécimale (minuscules).
%f	Rationnel en notation décimale (float)	Affiche un nombre flottant en notation décimale.
%e	Rationnel en notation scientifique (float)	Affiche un nombre flottant en notation scientifique.
%g	Rationnel en notation décimale/scientifique (float)	Affiche un nombre flottant, selon la forme la plus concise (décimale ou scientifique).
%lf	Rationnel en notation décimale (double)	Affiche un nombre à virgule flottante double en notation décimale.
%lg	Rationnel en notation décimale/scientifique (double)	Affiche un nombre double en notation décimale ou scientifique.
%le	Rationnel en notation scientifique (double)	Affiche un nombre double en notation scientifique.
%c	Caractère (char)	Affiche un caractère.
%s	Chaîne de caractères (char*)	Affiche une chaîne de caractères.

Les spécificateurs de format

Pour les entiers de type long :

Symbole	Type	Description
%ld	Entier long signé	Affiche un entier long signé (long int).
%li	Entier long signé	Même que %ld, affiche un entier long signé.
%lu	Entier long non signé	Affiche un entier long non signé (unsigned long).
%lo	Entier long en octal	Affiche un entier long en base octale.
%lx	Entier long en hexadécimal	Affiche un entier long en base hexadécimale (minuscules).

Pour les rationnels de type long double :

ymbole	Type	Description
%Lf	Rationnel long en notation décimale (long double)	Affiche un nombre long double en notation décimale.
%Lg	Rationnel long en notation décimale/scientifique (long double)	Affiche un nombre long double en notation décimale ou scientifique.
%Le	Rationnel long en notation scientifique (long double)	Affiche un nombre long double en notation scientifique.

Les spécificateurs de format

Largeur minimale des entiers

Il est possible d'indiquer la largeur minimale de la valeur à afficher. Les nombres sont alors alignés à droite dans l'espace réservé.

printf("%4d", 1); → v v v 1

printf("%4d", 123); → v 123

printf("%4d", 1234); → 1234

```c
#include <stdio.h>

int main()
{
printf("%d\n", 1);
printf("%4d\n", 1);
printf("%4d\n", 123);
printf("%4d", 1234);
    return 0;
}
```

```
1
   1
 123
1234

...Program finished with exit code 0
Press ENTER to exit console.
```

Largeur minimale et précision pour les rationnels

La précision par défaut est de 6 décimales.

printf("%f" , 12.34); → 12.340000

La syntaxe est: "%n.mf" où:

n est la largeur du champ

m est le nombre de décimales

Exemple

printf("%10.3f", 100.123); → **v v v 100.123**

printf("%10.2f", 100.123); → **v v v v 100.12**

Lecture formatée de données

scanf() : Permet de lire des valeurs depuis l'entrée standard (clavier) et de les stocker dans des variables en utilisant des spécificateurs de format similaires à ceux de **printf**().

Syntaxe:

scanf("format", adr_var_1, adr_var_2)

Format de lecture des données

Adresses des variables auxquelles les données sont attribuées

- La fonction **scanf** reçoit ses données à partir du fichier standard stdin (le clavier)
- La chaîne de format détermine comment les données reçues doivent être interprétées
- Les données reçues correctement sont mémorisées aux adresses indiquées par adr_var_1, adr_var_2, ...

Exemple

```c
#include <stdio.h>
int main() {
    int age;
    float taille;
    // Demander à l'utilisateur d'entrer
son âge et sa taille
    printf("Entrez votre âge : ");
    scanf("%d", &age);  // Lire un entier
    printf("Entrez votre taille (en mètres)
: ");
    scanf("%f", &taille);  // Lire un
nombre flottant
    // Afficher les valeurs saisies
    printf("Votre âge est : %d ans\n",
age);
    printf("Votre taille est : %.2f mètres\
n", taille);
    return 0;
}
```

Explication du code

scanf("%d", &age); : Lit un entier (**avec %d**) et le stocke dans la variable age.

scanf("%f", &taille); : Lit un nombre flottant (**avec %f**) et le stocke dans la variable taille.

printf("%d", age); : Affiche la valeur de age à l'écran.

printf("%.2f", taille); : Affiche la valeur de taille avec deux décimales.

Les spécificateurs de format pour scanf sont:

Symbole	Type	Description
%d ou %i	Entier relatif (int)	Lit un entier signé.
%u	Entier naturel (non signé) (unsigned int)	Lit un entier non signé.
%o	Entier exprimé en octal (int)	Lit un entier en base octale.
%x	Entier exprimé en hexadécimal (int)	Lit un entier en base hexadécimale (en minuscules).
%f	Rationnel en notation décimale (float)	Lit un nombre flottant en notation décimale.
%e	Rationnel en notation scientifique (float)	Lit un nombre flottant en notation scientifique.
%g	Rationnel en notation décimale/scientifique (float)	Lit un nombre flottant en notation décimale ou scientifique.
%lf	Rationnel en notation décimale (double)	Lit un nombre double en notation décimale.
%lg	Rationnel en notation décimale/scientifique (double)	Lit un nombre double en notation décimale ou scientifique.
%le	Rationnel en notation scientifique (double)	Lit un nombre double en notation scientifique.
%c	Caractère (char)	Lit un caractère.
%s	Chaîne de caractères (char*)	Lit une chaîne de caractères.

Remarque

On peut traiter plusieurs variables avec une seule instruction scanf: Lors de l'entrée des données, une suite de signes d'espacement (espace, tab, interligne) est évaluée comme un seul espace

Exemple

int jour, mois, annee;
scanf("%d %d %d",&jour, &mois, &annee);
Les entrées suivantes sont correctes et équivalentes
24 ∨ 11 ∨ 1973↵
24→11 → 1973↵
24↵
11↵
1973↵

Si la chaîne de format contient aussi d'autres caractères que des signes d'espacement, alors ces symboles doivent être introduits exactement dans l'ordre indiqué

Exemple

int jour, mois, annee;
scanf("%d/%d/%d",&jour, &mois, &annee);

Entrées correctes	**Entrées rejetées**
24/11/1973 ↵	24 ∨ 11 ∨ 1973 ↵
24/011/1973 ↵	24 ∨ /11/ ∨ 1973 ↵

Ecriture et Lecture d'un caractère

L'écriture d'un caractère en C peut être réalisée en utilisant les fonctions de base de la bibliothèque standard C

Exemple 1 : Affichage d'un caractère à l'écran.
Pour cela on utilise la fonction **putchar** pour afficher un caractère.

```c
#include <stdio.h>
int main() {
    char c = 'A'; // Déclaration d'un caractère
    putchar(c); // Affichage du caractère
    putchar('\n'); // Passage à la ligne
    return 0;
}
```

Exemple 2 : Lecture et écriture d'un caractère

```c
#include <stdio.h>
int main() {
    char c;
    printf("Entrez un caractère : ");
    c = getchar(); // Lecture du caractère
    printf("Vous avez entré : ");
    putchar(c); // Affichage du caractère
    putchar('\n'); // Passage à la ligne
    return 0;
}
```

Écriture d'un caractère dans un fichier

Pour écrire un caractère dans un fichier, on utilise la fonction fputc

Exemple :

```
#include <stdio.h>  //  permet d'utiliser des fonctions d'entrée/sortie standard.

int main() {  //début de l'exécution du programme.
   FILE *file = fopen("output.txt", "w"); // Ouverture d'un fichier nommé "output.txt" en mode écriture, Si le fichier n'existe pas, il sera créé. La variable file est un pointeur vers un objet FILE.
   if (file == NULL) { // Vérifie si l'ouverture a échoué
      printf("Erreur d'ouverture du fichier\n"); // Affiche un message d'erreur
      return 1; // Retourne 1 pour indiquer une erreur
   }

   char c = 'C'; // Déclaration d'un caractère 'C' . Le caractère C est stocké dans la variable c.
   fputc(c, file); // Écrit le caractère 'C' dans le fichier output.txt
   fclose(file);   // Ferme le fichier pour libérer les ressources associées.
   printf("Le caractère '%c' a été écrit dans le fichier output.txt\n", c); // Affiche un message confirmant l'écriture
   return 0; // Retourne 0 pour indiquer que le programme s'est terminé avec succès
}
```

V – Structure alternative

En langage C, une structure alternative fait référence aux instructions conditionnelles qui permettent de prendre des décisions basées sur des conditions. La structure alternative la plus utilisée est l'instruction **if**, mais d'autres constructions comme **switch** sont également courantes.

1- Instruction if

Syntaxe :

```
if (condition) {
// Code à exécuter si la condition est vraie
}
```

L'instruction **if** permet d'exécuter un bloc de code seulement si une condition est vraie.

Exemple 1

```c
int age = 20;
if (age >= 18) {
    printf("Vous êtes majeur.\n");
}
```

Exemple 2

```c
#include <stdio.h>
int main() {
    int nombre;

    printf("Entrez un nombre entier :
");
    scanf("%d", &nombre);

    // Vérification si le nombre est
pair
    if (nombre % 2 == 0) {
        printf("Le nombre %d est
pair.\n", nombre);
    }
    // Fin du programme
    printf("Programme terminé.\n");
    return 0;
```

2- Instruction if...else

L'instruction **if...else** permet de définir un chemin alternatif si la condition est fausse.

Syntaxe :

```
if (condition) {
   // Code à exécuter si la condition est vraie
} else {
   // Code à exécuter si la condition est fausse
}
```

Exemple

```c
#include <stdio.h>
int main() {
    int age = 16;
        if (age >= 18)
        {
            printf("Vous êtes majeur.\
n");
        }
        else
        {
            printf("Vous êtes mineur.\
n");
        }
    return 0;
}
```

3- Instruction if-else imbriqués

Utilisée pour tester plusieurs conditions successives.

Syntaxe :

```
if (condition1) {
    // Code si condition1 est vraie
} else if (condition2) {
    // Code si condition2 est vraie
} else {
    // Code si aucune condition
n'est vraie
}
```

Exemple

```
#include <stdio.h>
int main() {
    float note;

    printf("Entrez une note :");
    scanf("%f", &note);
if (note >= 18) {
    printf("Excellent\n");
} else if (note >= 16) {
    printf("T. Bien\n");
} else if (note >= 14) {
    printf("Bien\n");
}else if (note >= 12) {
    printf("A. Bien\n");
} else if (note >= 10) {
    printf("Passable\n");
} else {
    printf("Rattrapage\n");
}
    return 0;
}
```

Remarque

Un **else** est toujours rattaché au dernier if qui ne possède pas de **else**

Exemple 1 : le else est donc rattaché au if (A>B)

```
if(N>0)
    if(A>B)
        MAX=A;
    else
        MAX=B;
```

Exemple 2 : forcer le programme à la seconde interprétation : Le else est rattaché au premier if

```
if(N>0)
        {
        if(A>B)
            MAX=A;
        }
else MAX=B;
```

Exemple 3

```
#include <stdio.h>
int main() {
   int A,B ;
printf("Entrer deux nombres entiers:\
n") ;
scanf("%d %d",&A,&B);
if(A>B)
printf("%d est plus grand que %d\
n",A,B);
else
if(A<B)
printf("%d est plus petit que %d\
n",A,B);
else
printf("%d est égal à %d\n",A,B);
```

Exemple 4

```
#include <stdio.h>
int main() {
    char C;
    printf("Entrez une commande (O ou N) : ");
    C = getchar();
    if (C == 'O') {
        printf("Commande acceptée : Option O\
n");
    }
    else if (C == 'N') {
        printf("Au revoir…\n");
    }
    else {
        printf("Erreur d'entrée\n");
    }
    return 0;
}
```

Exemple 4

```c
#include <stdio.h>
int main() {
    char C;
    printf("Entrez une commande (O ou N) : ");
    C = getchar();
    if (C == 'O') {
        printf("Commande acceptée : Option O\
n");
    }
    else if (C == 'N') {
        printf("Au revoir…\n");
    }
    else {
        printf("Erreur d'entrée\n");
    }
    return 0;
}
```

Cas 1 : L'utilisateur entre 'O'

Entrez une commande (O ou N) : O
Commande acceptée : Option O

Cas 2 : L'utilisateur entre 'N'

Entrez une commande (O ou N) : N
Au revoir…

Cas 3 : L'utilisateur entre un autre caractère, comme 'X'

Entrez une commande (O ou N) : X
Erreur d'entrée

4- Instruction switch

Permet de choisir parmi plusieurs cas possibles en fonction de la valeur d'une variable.

Syntaxe

```
switch (variable) {
    case valeur1:
        // Code si variable == valeur1
        break;
    case valeur2:
        // Code si variable == valeur2
        break;
    // ...
    default:
        // Code si aucune valeur ne correspond
        break;
}
```

Exemple

```
#include <stdio.h>
int main() {
    int jour;

    // Demander à l'utilisateur d'entrer un numéro pour un jour de la semaine
    printf("Entrez un numéro (1 à 7) pour un jour de la semaine : ");
    scanf("%d", &jour);

    // Utilisation de switch pour afficher le jour correspondant
    switch (jour) {
        case 1:
            printf("Lundi\n");
            break;
        case 2:
            printf("Mardi\n");
            break;
        case 3:
            printf("Mercredi\n");
            break;
        case 4:
            printf("Jeudi\n");
            break;
        case 5:
            printf("Vendredi\n");
            break;
        case 6:
            printf("Samedi\n");
            break;
        case 7:
            printf("Dimanche\n");
            break;
        default:
            printf("Jour inconnu\n");
            break;
    }
    return 0;
}
```

Utilisation de if else

```c
#include <stdio.h>
int main() {
    int jour;

    // Demander à l'utilisateur d'entrer un numéro pour un jour de la semaine
    printf("Entrez un numéro (1 à 7) pour un jour de la semaine : ");
    scanf("%d", &jour);

    // Utilisation de if-else pour afficher le jour correspondant
    if (jour == 1) {
        printf("Lundi\n");
    } else if (jour == 2) {
        printf("Mardi\n");
    } else if (jour == 3) {
        printf("Mercredi\n");
    } else if (jour == 4) {
        printf("Jeudi\n");
    } else if (jour == 5) {
        printf("Vendredi\n");
    } else if (jour == 6) {
        printf("Samedi\n");
    } else if (jour == 7) {
        printf("Dimanche\n");
    } else {
        printf("Jour inconnu\n");
    }
    return 0;
}
```

Utilisation de switch

```c
#include <stdio.h>
int main() {
    int jour;

    // Demander à l'utilisateur d'entrer un numéro pour un jour de la semaine
    printf("Entrez un numéro (1 à 7) pour un jour de la semaine : ");
    scanf("%d", &jour);

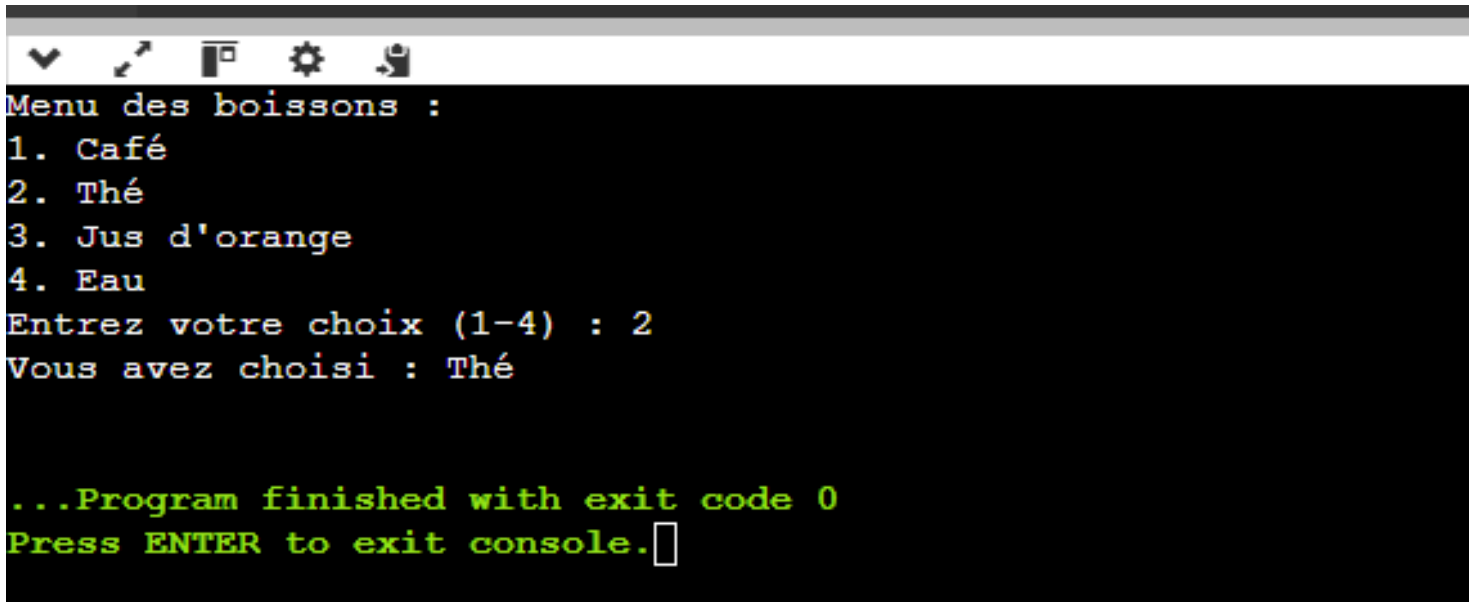
    // Utilisation de switch pour afficher le jour correspondant
    switch (jour) {
        case 1:
            printf("Lundi\n");
            break;
        case 2:
            printf("Mardi\n");
            break;
        case 3:
            printf("Mercredi\n");
            break;
        case 4:
            printf("Jeudi\n");
            break;
        case 5:
            printf("Vendredi\n");
            break;
        case 6:
            printf("Samedi\n");
            break;
        case 7:
            printf("Dimanche\n");
            break;
        default:
            printf("Jour inconnu\n");
            break;
    }
    return 0;
}
```

Exercice d’application

Application 1 : Écrire un programme qui permet de choisir une boisson en fonction d'un numéro saisi par l'utilisateur, en utilisant l'instruction switch.

Application 2 : Réécrire le même programme en utilisant l'instruction if-else

L'exécution du programme doit ressembler à :



Correction

Instruction if-else

```c
#include <stdio.h>
int main() {
    int choix;
    // Afficher le menu des boissons
    printf("Menu des boissons :\n");
    printf("1. Café\n");
    printf("2. Thé\n");
    printf("3. Jus d'orange\n");
    printf("4. Eau\n");
    printf("Entrez votre choix (1-4) : ");
    scanf("%d", &choix);
    // Utilisation de if-else pour afficher la boisson
choisie
    if (choix == 1) {
        printf("Vous avez choisi : Café\n");
    } else if (choix == 2) {
        printf("Vous avez choisi : Thé\n");
    } else if (choix == 3) {
        printf("Vous avez choisi : Jus d'orange\n");
    } else if (choix == 4) {
        printf("Vous avez choisi : Eau\n");
    } else {
        printf("Choix invalide, veuillez choisir entre
1 et 4.\n");
    }
    return 0;
}
```

Instruction switch

```c
#include <stdio.h>
int main() {
    int choix;
   // Afficher le menu des boissons
    printf("Menu des boissons :\n");
    printf("1. Café\n");
    printf("2. Thé\n");
    printf("3. Jus d'orange\n");
    printf("4. Eau\n");
    printf("Entrez votre choix (1-4) : ");
    scanf("%d", &choix);

    // Utilisation de switch pour afficher la boisson choisie
    switch (choix) {
        case 1:
            printf("Vous avez choisi : Café\n");
            break;
        case 2:
            printf("Vous avez choisi : Thé\n");
            break;
        case 3:
            printf("Vous avez choisi : Jus d'orange\n");
            break;
        case 4:
            printf("Vous avez choisi : Eau\n");
            break;
        default:
            printf("Choix invalide, veuillez choisir entre 1
et 4.\n");
            break;
    }
    return 0;
}
```

6 - Structures répétitives

Les **structures répétitives** (ou boucles) sont essentielles en langage C pour exécuter un bloc d'instructions de manière répétée, tant qu'une condition est remplie. Elles permettent de gérer efficacement des tâches répétitives.

1- La boucle while = tant que

Elle exécute un bloc d'instructions tant que la condition est vraie (non nulle). Si la condition est fausse dès le départ, la boucle ne s'exécute jamais.

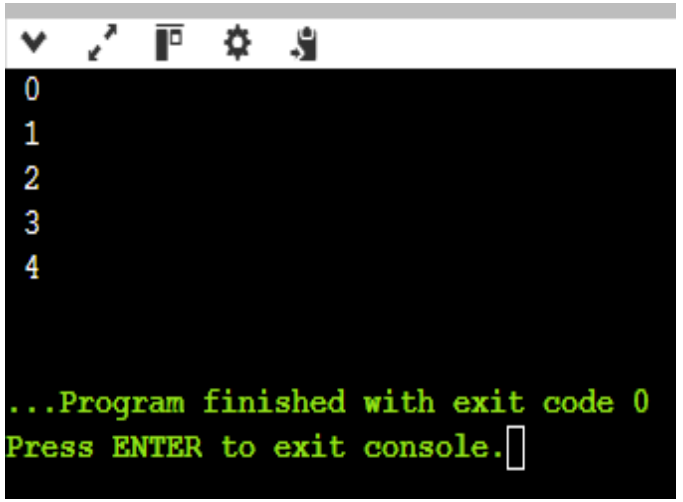
Syntaxe

```
while (condition)
    {
    // Instructions à répéter
    }
```

Exemple

```
#include <stdio.h>
int main() {
    int compteur = 0;
    while (compteur < 5) {
        printf(" %d\n",
compteur);
        compteur++;
    }
    return 0;
}
```

Exécution



6 - Structures répétitives

Remarque : Dans le cas d'une seule instruction, les accolades sont inutiles

Exemple 1:

```
int i =0;
while(i<4)
        {
        printf("%d\n",i);
        i++ ;
        }
```

Affichage:
0
1
2
3

Exemple 2:

```
int i =0;
while(i<4)
        printf("%d\n",i++);
```

Affichage:
0
1
2
3

Exemple 3:

```
int i =0;
while(i<4)
        printf("%d\n",++i);
```

Affichage:
1
2
3
4

Exemple 4:

```
int i =4;
while(i)
        printf("%d\n",i--);
```

Affichage:
4
3
2
1

while (i) // La condition de la boucle est que i est non nul

i-- est l'opérateur de post-décrément, ce qui signifie que la valeur de i est d'abord utilisée dans l'expression (affichée avec printf), puis i est décrémenté après l'affichage.

6 - Structures répétitives

2- La boucle do...while : faire - tant que

Similaire à **while**, mais elle s'assure que le bloc d'instructions est exécuté au moins une fois avant de tester la condition.

Syntaxe

```
do // on exécute
        {
        // bloc instructions
        }
        while (condition); tant que condition est vrai
(≠0)
```

Exemple

```
#include <stdio.h>
int main() {
    int compteur = 0;
    do {
        printf("Compteur : %d\n",
compteur);
        compteur++;
    } while (compteur < 5);
    return 0;
}
```

Exécution

```
Compteur : 0
Compteur : 1
Compteur : 2
Compteur : 3
Compteur : 4


...Program finished with exit code 0
Press ENTER to exit console.
```

2- La boucle do...while : faire - tant que

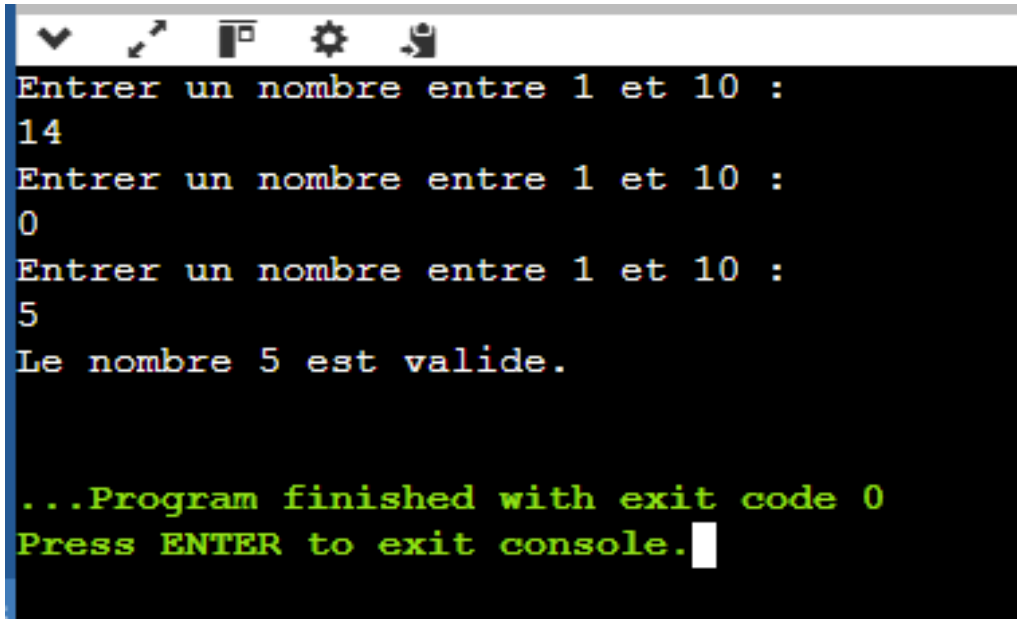
Exemple 2 : Ecrire un programme qui demande à l'utilisateur de saisir un nombre entre 1 et 10, et qui répète cette demande tant que le nombre saisi est en dehors de cette plage.

```c
#include <stdio.h>
int main() {
    int n;
    // Demande de saisir un nombre entre 1 et 10
    do {
        printf("Entrer un nombre entre 1 et 10 :\
n");
        scanf("%d", &n);  // Lecture du nombre
saisi
    } while (n < 1 || n > 10);  // Condition de
répétition

    // Affichage du nombre valide
    printf("Le nombre %d est valide.\n", n);

    return 0;
}
```

Exécution



2- La boucle for

Elle est utilisée lorsque le nombre d'itérations est connu à l'avance. Elle permet d'initialiser une variable, de tester une condition, et de mettre à jour la variable dans une structure compacte.

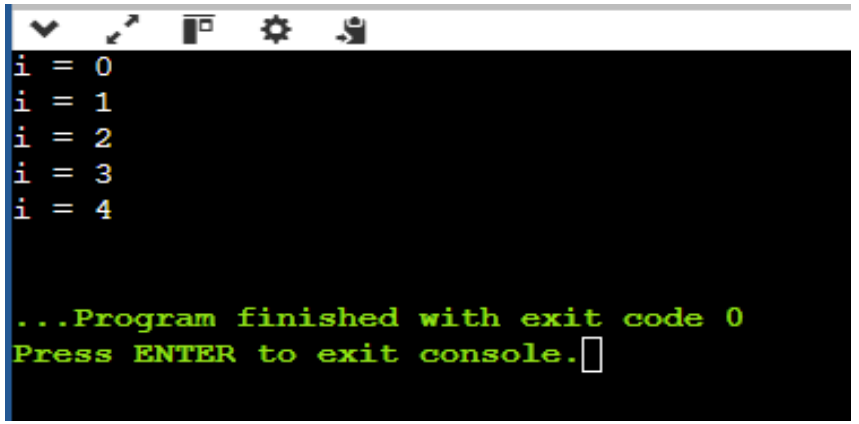
Syntaxe

```
for (initialisation; condition; mise à
jour)
       {
        // Instructions à répéter
       }
```

Exemple

```c
#include <stdio.h>
int main() {
    for (int i = 0; i < 5; i++) {
        printf("i = %d\n", i);
    }
    return 0;
}
```

Exécution



2- La boucle for

Exemple 2 : Ecrire un programme qui affiche les carrés des nombres de 1 à 4 dans un format clair

```c
#include <stdio.h>
int main() {
    int i;

    // Boucle pour afficher les carrés des nombres
de 1 à 4
    for(i = 1; i <= 4; i++) {
        printf("Le carré de %d est %d\n", i, i *
i);
    }
    return 0;
}
```

Rendu

```
Le carré de 1 est 1
Le carré de 2 est 4
Le carré de 3 est 9
Le carré de 4 est 16
```

Choix de la structure répétitive

- for : Quand le nombre d'itérations est connu ou fixe.
- while : Quand on ne sait pas combien de fois la boucle sera exécutée à l'avance, mais nous avons une condition qui détermine la fin de la boucle.
- do…while : Quand on veut s’assurer que le bloc d'instructions s'exécute au moins une fois, indépendamment de la condition.

Chaque structure a ses avantages, et le choix dépendra des exigences spécifiques du programme.

VII – Les tableaux

Les tableaux en langage C sont des **structures de données** qui permettent de stocker plusieurs éléments **du même type** en mémoire, sous **un même nom.**

A – Tableaux à une dimension

Déclaration et initialisation

Un tableau peut être statiquement ou dynamiquement alloué en C

Tableau statique:

1 **Déclaration**

La déclaration d'un tableau statique est faite en spécifiant le type des éléments, le nom du tableau, et sa taille

```
type nomTableau[taille];
```

2 **Initialisation**

Un tableau peut être initialisé lors de sa déclaration :

```
int nombres[5] = {1, 2, 3, 4, 5}; // Tableau d'entiers
float notes[3] = {12.5, 15.0, 9.8}; // Tableau de flottants
char voyelles[5] = {'a', 'e', 'i', 'o', 'u'}; // Tableau de caractères
```

3 Exemple

```
int nombres[5]; // Tableau de 5 entiers
float notes[10]; // Tableau de 10 flottants
char lettres[26]; // Tableau de 26 caractères
```

4 **Cas particulier lors de l'initialisation**

Si tous les éléments ne sont pas fournis lors de l'initialisation, les composantes restantes seront initialisés à **zéro (pour les types numériques)** ou à **\0 (pour les caractères) :**

Taille implicite d'un tableau

Si la taille n'est pas indiquée, le compilateur réserve automatiquement le nombre d'octets nécessaires.

Exemple `int nombres[] = {1, 2, 3, 4, 5};`

Accès aux éléments

Les éléments d'un tableau sont accessibles en utilisant leur indice, qui commence à zéro :

Exemple

```
nombres[0] = 10; // Premier élément
printf("%d\n", nombres[0]); // Affiche
10
```

Affichage et affectation

1 Affichage du contenu d'un tableau

```
#include<stdio.h>
int main()
{
int A[5]={1,2,3,4,5} ;
int i ;
for(i=0 ;i<5 ;i++)
printf("%3d",A[i]);
return 0;
}
```

```
  1  2  3  4  5
PS D:\testc>
```

2 Affectation

De la même manière que pour une variable "normale", on indique l'adresse **(&A[i])**

```
#include<stdio.h>
int main()
{
int A[5]={1,2,3,4,5} ;
int i ;
for(i=0 ;i<5 ;i++)
scanf("%d",&A[i]);
printf("Les nouvelles valeurs introduites sont\
n");
for(i=0;i<5;i++)
printf("%d ", A[i]);
return 0;
}
```

```
Les nouvelles valeurs introduites sont
45 46 47 48 49
```

B – Tableaux à deux dimensions

Les tableaux peuvent avoir plusieurs dimensions, souvent utilisés pour représenter des matrice, **avec des lignes** et **des colonnes**. Chaque élément d'un tableau à deux dimensions est repéré par deux indices **: l'indice de la ligne** et **celui de la colonne.**

Déclaration et initialisation

1 Déclaration

```
type Nom_du_tableau[nbrDeLignes][nbrDeColonnes];

int matrice[3][4]; // Tableau de 3 lignes et 4 colonnes
```

2 Initialisation

```
int matrice[2][3] = {
 {1, 2, 3},  // Première ligne
 {4, 5, 6}   // Deuxième ligne
};
```

```
int matrice[][3] =  {
 {1, 2, 3},
  {4, 5, 6}
 };
```

Si la taille de la première dimension n'est pas spécifiée, le compilateur la déduit automatiquement :

3 Accès aux éléments

Les éléments du tableau sont accessibles en utilisant deux indices :
- Le premier indice représente la ligne.
- Le deuxième indice représente la colonne.

En C, la **taille de toutes les dimensions sauf la première** doit être spécifiée lors de la déclaration d'un tableau multidimensionnel.

```c
int matrice[2][3] = {{1, 2, 3}, {4, 5, 6}};
printf("%d\n", matrice[0][1]); // Affiche l'élément à la 1ère ligne, 2ème colonne (2)
matrice[0][1] = 10;
printf("la nouvelle valeur modifiée est :\n");
printf("%d\n",matrice[0][1]);// Affiche le nouvel élément à la 1ère ligne, 2ème colonne
return 0;
```

```
2
la nouvelle valeur modifiee est :
10
```

Exemple : Affichage et affectation

Exemple 1

```c
#include<stdio.h>
int main()
{
int A[3][2]= {{1,2},{10,20},{100,200}};
int i=2, j=1;
printf("Element [%d][%d] = %d",i,j,A[i][j]);
return 0;
}
```

```
Element [2][1] = 200
PS D:\testc>
```

Exemple 2

```c
#include <stdio.h>

int main() {
    // Déclaration et initialisation d'un tableau 2D
    int A[3][2] = {{1, 2}, {10, 20}, {100, 200}};
    int i, j;

    // Parcours des lignes et colonnes pour afficher
les éléments
    printf("Affichage de la matrice :\n");
    for (i = 0; i < 3; i++) { // Parcours des lignes
        for (j = 0; j < 2; j++) { // Parcours des
colonnes
            printf("%5d", A[i][j]); // Affiche chaque
élément avec un format aligné
        }
        printf("\n"); // Nouvelle ligne après chaque
ligne de la matrice
    }

    // Exemple d'accès à un élément spécifique
    int ligne = 1, colonne = 0; // Accès à la deuxième
ligne, première colonne
    printf("\nElement [%d][%d] = %d\n", ligne, colonne,
A[ligne][colonne]);

    return 0;
}
```

```
Affichage de la matrice :
    1    2
   10   20
  100  200
```

Exemple : Application

Ecrire un programme qui demande à l'utilisateur
1- de saisir les valeurs d'une matrice 3x2, puis l'affiche.
2- d'accéder et d'afficher un élément spécifique de la matrice en fonction des indices fournis par l'utilisateur.

```
Saisissez les valeurs pour une matrice 3x2 :
Element [0][0] : 1
Element [0][1] : 2
Element [1][0] : 3
Element [1][1] : 4
Element [2][0] : 5
Element [2][1] : 6

Affichage de la matrice :
    1    2
    3    4
    5    6

Entrez l'indice de la ligne (0-2) : 1
Entrez l'indice de la colonne (0-1) : 1

Element [1][1] = 4
```

```c
#include <stdio.h>

int main() {
    // Déclaration d'un tableau 2D
    int A[3][2];
    int i, j;

    // Saisie des valeurs du tableau
    printf("Saisissez les valeurs pour une matrice 3x2 :\n");
    for (i = 0; i < 3; i++) { // On parcours des lignes
        for (j = 0; j < 2; j++) { // On arcours des colonnes
            printf("Element [%d][%d] : ", i, j);
            scanf("%d", &A[i][j]); // Saisie de l'utilisateur
        }
    }

    // Affichage du tableau
    printf("\nAffichage de la matrice :\n");
    for (i = 0; i < 3; i++) { // On arcours des lignes
        for (j = 0; j < 2; j++) { // On parcours des colonnes
            printf("%5d", A[i][j]); // Affichage formaté
        }
        printf("\n"); // Nouvelle ligne après chaque ligne de la
matrice
    }

    // Exemple d'accès à un élément spécifique
    int ligne, colonne;
    printf("\nEntrez l'indice de la ligne (0-2) : ");
    scanf("%d", &ligne);
    printf("Entrez l'indice de la colonne (0-1) : ");
    scanf("%d", &colonne);

    printf("\nElement [%d][%d] = %d\n", ligne, colonne, A[ligne]
[colonne]);

    return 0;
}
```

VIII – Les chaînes de caractères

En C, une chaîne de caractères est un tableau de caractères, terminé par un caractère nul **\0**. Ce caractère de fin permet de signaler la fin de la chaîne. Les chaînes sont manipulées comme des tableaux de caractères.

Une chaîne de caractères est traitée comme un tableau de caractères à une dimension.

1 Déclaration et initialisation

Lors de la déclaration (comme pour les tableaux de chiffres), on doit indiquer l'espace à réserver.

```
char Nom_du_tableau[longueur] ;
```

Exemple

```
char TXT[9]="BONJOUR!";
```

Ici, "Bonjour" est une chaîne de caractères qui est automatiquement terminée par **\0**.
Cela signifie que le tableau **TXT** contient en réalité les éléments suivants :



TXT
TXT[0]

le nom de la chaîne est le représentant de son adresse mémoire, et plus précisément le représentant de l'adresse du premier caractère.

```
char chaine1[] = "Bonjour"; // Taille déterminée automatiquement
char chaine2[10] = "Salut"; // Taille définie manuellement
char chaine3[10] = {'H', 'e', 'l', 'l', 'o', '\0'}; // Avec le caractère de fin explicitement
```

2 Les chaînes de caractères constantes

Les chaînes de caractères sont indiquées entre guillemets. La chaîne de caractères vide est alors:""

Remarques

Dans les chaînes de caractères, il est possible d'utiliser des séquences d'échappement, qui sont définies comme des caractères constants.

Exemple

```c
#include <stdio.h>
int main() {
    char C[] = "Ce texte\n s'ecrira sur\n 3
lignes";
    printf("%s", C);
    return 0;
}
```

```
PS D:\testc> cd "d:\testc\" ; if ($?) { gcc Car1.c -o Car1 } ; if ($?) { .\Car1 }
Ce texte
 s'ecrira sur
 3 lignes
PS D:\testc>
```

Exemple

```c
#include <stdio.h>

int main() {
   // Définition de la chaîne de caractères
   char txt[] = {'L', '\'', 'a', 's', 't', 'u', 'c', 'e', '\0'}; // chaîne "L'as
tu ce"

   // Affichage de la chaîne avec printf
   printf("%s", txt);

   return 0;
}
```

```
PS D:\testc> cd "d:\testc\" ; if ($?) { gcc Car1.c -o Car1 } ; if ($?) { .\Car1 }
L'astuce
PS D:\testc>
```

Rappel sur quelques séquences d'échappement courantes en C

\n : Saut de ligne (newline)

```
 printf("Bonjour\nMonde\n");  // Ajout du second \n
pour ajouter un retour à la ligne après "Monde"
```

```
Bonjour
Monde
```

\\ : Backslash (Représente un caractère backslash (\))

```
printf("Chemin du fichier : C:\\Program Files");
```

```
Chemin du fichier : C:\Program Files
```

\' : Apostrophe (guillemet simple) Permet d'insérer une apostrophe dans une chaîne de caractères.

```
  printf("L'apostrophe s'echappe : \'");
```

```
L'apostrophe s'echappe : '
```

\" : Guillemet double

```
Ceci est une "citation"
```

\0 : Caractère nul

```
char str[] = "Bonjour\0Monde";
printf("%s", str);
```

```
Bonjour
```

La fonction **printf** avec le spécificateur **%s** affiche la chaîne jusqu'au premier caractère nul **(\0)**, qu'elle rencontre

\r : Retour chariot

```
printf("Hello\rWorld")
```

```
World
```

replace le curseur au début de la ligne sans créer une nouvelle ligne.

Les séquences d'échappement en C permettent d'inclure des caractères spéciaux dans une chaîne de caractères, qui ne peuvent pas être représentés directement (comme les guillemets, les retours à la ligne, les tabulations, etc.). Cela rend la manipulation des chaînes plus flexible et puissante.

Remarque

Plusieurs chaînes de caractères constantes qui sont séparées par des signes d'espacement dans le texte du programme seront réunies en une seule chaîne lors de la compilation:

Exemple

char TXT[]="un ""deux ""trois";

```
char TXT[]="un ""deux ""trois";
printf("%s", TXT);
```

```
un deux trois
```

'x' est un caractère constant (qui a une valeur numérique 'x'=120) (codé sur 1 octet)
"x" est une chaîne de caractères qui contient 2 caractères 'x' et '\0' (codé sur 2 octets)

3 Représentation en mémoire

char txt[]="hello";

'H'	'e'	'l'	'l'	'o'	'\0'

char txt[6]="hello";

'H'	'e'	'l'	'l'	'o'	'\0'

char txt[8]="hello"

'H'	'e'	'l'	'l'	'o'	'\0'	'\0'	'\0'

char txt[5]="hello";

'H'	'e'	'l'	'l'	'o'

Compilation et exécution réussies selon la configuration du compilateur

char txt[4]="hello";

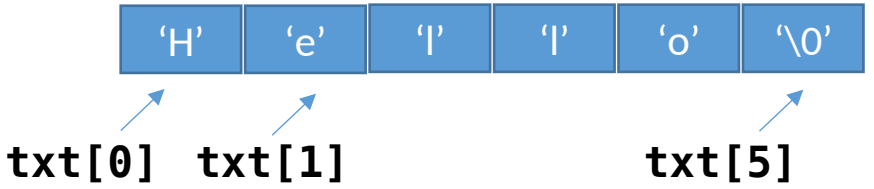
'H'	'e'	'l'	'l'

erreur de compilation dans la plupart des compilateurs, car il y a un dépassement de mémoire

4 Accès aux éléments d'une chaîne

On peut accéder aux éléments d'une chaîne de caractères de la même manière que pour un tableau classique, en utilisant un indice.

Exemple




```c
#include <stdio.h>
int main() {
    char txt[] = "hello";

    // Accéder aux éléments de la chaîne
    printf("Premier caractere: %c\n", txt[0]);  // 'h'
    printf("Deuxieme caractere: %c\n", txt[1]); // 'e'
    printf("Troisieme caractere: %c\n", txt[2]); // 'l'
    printf("Quatrieme caractere: %c\n", txt[3]); // 'l'
    printf("Cinquieme caractere: %c\n", txt[4]); // 'o'
    printf("Caractere nul: %c\n", txt[5]); // '\0'
    return 0;
}
```

```
Premier caractere: h
Deuxieme caractere: e
Troisieme caractere: l
Quatrieme caractere: l
Cinquieme caractere: o
Caractere nul:
```

5 **manipulation des chaînes**

I- Les fonctions de <stdio.h>

Affichage

A- **printf** (avec le spécificateur **%s**)

```
char NOM[]="Hello
world";
```

```
printf(":%s:",NOM);
```

```
:Hello world:
```

```
printf(":%15s:",NOM)
```

```
:    Hello world:
```

(alignement à droite)

```
printf(":%-15s:",NOM);
```

```
:Hello world    :
```

(alignement à gauche)

B- **puts** (avec le spécificateur **%s**)

est une fonction simple et pratique pour afficher une chaîne de caractères suivie d'un saut de ligne

Equivalence: **puts(ch) = printf("%s\n",ch)**

Exemple d'une mauvaise utilisation

```
    char chaine[] = {'H', 'e', 'l', 'l', 'o'};  // Pas de
'\0'
    puts(chaine);  // Comportement indéfini !
```

```
// Comportement indéfini !
```

puts() n'a pas de limite claire et continue à parcourir la mémoire jusqu'à ce qu'elle rencontre un **'\0'** par hasard. **Tout dépend du compilateur**

Avantages et Inconvénients

puts() est simple et conçu pour afficher uniquement des chaînes.
printf() est plus puissant et flexible pour formater la sortie avec des spécificateurs (%d, %f, etc.).

Lecture



Le nom de la chaîne est le représentant de l'adresse du premier caractère, il est donc inutile (et même interdit par certains compilateurs) de mettre &.

A- Utilisation de **scanf()**

```c
#include <stdio.h>

int main() {
    char chaine[100];
    printf("Entrez une chaîne : ");
    scanf("%s", chaine);  // Lecture de la chaîne sans espaces
    printf("Vous avez entré : %s\n", chaine);

    return 0;
}
```

```
Veuillez entrer une chaine :
ma chaine
la chaine que vous avez entré est : ma
```

Caractéristiques :

- ❑ Lit une chaîne jusqu'au premier espace, tabulation ou saut de ligne (\n).
- ❑ Ne fonctionne pas pour des chaînes contenant des espaces.
- ❑ Pas de gestion explicite des débordements

B- Utilisation de **gets()**

```c
#include <stdio.h>

int main() {
    char chaine[5];
    printf("Entrez une chaîne : ");
    gets(chaine);  // Lecture d'une ligne entière
    printf("Vous avez entré : %s\n", chaine);

    return 0;
}
```

Caractéristiques :

- ❑ Lit une ligne complète (inclut les espaces) jusqu'au saut de ligne.
- ❑ Remplacée par **fgets()** dans les standards modernes

Lecture

C- Utilisation de **fgets()**

```c
#include <stdio.h>

int main() {
    char chaine[10];
    printf("Entrez une chaîne : ");
    fgets(chaine, 10, stdin);  // Lecture
jusqu'à 9 caractères + '\0'
    printf("Vous avez entré : %s\n", chaine);
    return 0;
}
```

Caractéristiques :

- Lit jusqu'à un certain nombre de caractères, ou jusqu'à un saut de ligne (\n)
- Contrôle la taille de l'entrée

- `chaine` : Un tableau de caractères où la chaîne lue sera stockée
- `10` : Le nombre maximal de caractères à lire, y compris le caractère nul de fin de chaîne ('\0').
- `stdin` : Le flux d'entrée, ici c'est l'entrée standard (le clavier)

II- Les fonctions de <string.h>

Il existe plusieurs fonctions standard dans **<string.h>** pour manipuler les chaînes de caractères.

1- strlen() : obtenir la longueur d'une chaîne

```c
#include <stdio.h>
#include <string.h>

int main() {
    char txt[] = "Bonjour";
    printf("La longueur de la chaîne est: %lu\n",
strlen(txt)); // 7
    return 0;
}
```

2- strcpy() : copier une chaîne

```c
#include <stdio.h>
#include <string.h>

int main() {
    char txt1[] = "Bonjour";
    char txt2[10];

    strcpy(txt2, txt1);  // Copie de "Bonjour" dans
txt2
    printf("txt2: %s\n", txt2);  // Bonjour
    return 0;
}
```

```
txt2: Bonjour
```

3- strcat() : concaténer deux chaînes

```c
#include <stdio.h>
#include <string.h>

int main() {
    char txt1[20] = "Bonjour";
    char txt2[] = " tout le monde";
    strcat(txt1, txt2);  // Concatène " tout le monde" à
"Bonjour"
    printf("txt1: %s\n", txt1);  // Bonjour tout le monde

    return 0;
}
```

4- strcmp() : comparer deux chaînes

```c
#include <stdio.h>
#include <string.h>

int main() {
    char txt1[] = "Bonjour";
    char txt2[] = "Bonjour";
    char txt3[] = "Bonsoir";
    printf("Comparaison txt1 et txt2: %d\n", strcmp(txt1, txt2)); // 0 (égales)
    printf("Comparaison txt1 et txt3: %d\n", strcmp(txt1, txt3)); //< 0 (txt1 <
txt3)

    return 0;
}
```

```
Comparaison txt1 et txt2: 0
Comparaison txt1 et txt3: -1
```

3- strcat() : concaténer deux chaînes

```c
#include <stdio.h>
#include <string.h>

int main() {
    char txt1[20] = "Bonjour";
    char txt2[] = " tout le monde";
    strcat(txt1, txt2);  // Concatène " tout le monde" à
"Bonjour"
    printf("txt1: %s\n", txt1);  // Bonjour tout le monde

    return 0;
}
```

4- strcmp() : comparer deux chaînes

```c
#include <stdio.h>
#include <string.h>

int main() {
    char txt1[] = "Bonjour";
    char txt2[] = "Bonjour";
    char txt3[] = "Bonsoir";
    printf("Comparaison txt1 et txt2: %d\n", strcmp(txt1, txt2)); // 0 (égales)
    printf("Comparaison txt1 et txt3: %d\n", strcmp(txt1, txt3)); //< 0 (txt1 <
txt3)

    return 0;
}
```

```
Comparaison txt1 et txt2: 0
Comparaison txt1 et txt3: -1
```

III- Les fonctions de <stdlib.h>

La bibliothèque **<stdlib.h>** contient des déclarations de fonctions pour la conversion de nombres en chaînes de caractères et vice-versa.

A- Chaîne -> nombre

Fonction	Description
atoi(s)	Convertit une chaîne de caractères en entier (int).
atol(s)	Convertit une chaîne de caractères en entier long (long).
atof(s)	Convertit une chaîne de caractères en nombre à virgule flottante (double).

atoi(s)

```c
#include <stdlib.h>
#include <stdio.h>

int main() {
    char s[] = "123";  // Déclare une chaîne contenant un nombre
    int y;
    int x = atoi(s);   // Convertit la chaîne en entier et affecte la valeur à x
    y=x+2; // Affecte x + 2 à y
    printf("%d\n", x);   // Affiche le résultat sous forme d'entier
    printf("la valeur de y est : %d",y); // Affiche la valeur de y
    return 0;
}
```

```
123
la valeur de y est : 125
```

atol(s) Convertit une chaîne de caractères en entier long (long)

```c
#include <stdlib.h>
#include <stdio.h>

int main() {
    char s[] = "123456789"; // Déclare une chaîne contenant un nombre
    long x = atol(s); // Convertit la chaîne en nombre long
    printf("La valeur de x est : %ld\n", x); // Affiche la valeur de x
    return 0;
}
```

```
La valeur de x est : 123456789
```

atof(s) Convertit une chaîne de caractères en nombre à virgule flottante (double).

```c
#include <stdlib.h>
#include <stdio.h>

int main() {
    char s[] = "123.456";  // Déclare une chaîne contenant un nombre à virgule flottante
    double x = atof(s);    // Convertit la chaîne en nombre à virgule flottante
    printf("La valeur de x est : %.3f\n", x);
 // Affiche la valeur de x
    return 0;
}
```

```
La valeur de x est : 123.456
```

Règles générales pour la conversion:

- ❑ Les espaces au début d'une chaîne sont ignorés
- ❑ Il n'y a pas de contrôle du domaine de la cible
- ❑ La conversion s'arrête au premier caractère non-convertible
- ❑ Pour une chaîne non-convertible, les fonctions retournent zéro

B- Nombre -> chaîne

Les fonctions itoa, ltoa, et ultoa sont des fonctions de conversion d'entiers en chaînes de caractères dans le langage C. Elles convertissent un nombre dans un format spécifié et le stockent dans une chaîne de caractères.

itoa (n_int, s, b) Convertit un entier (n_int) en une chaîne de caractères.

> n_int : L'entier à convertir
> s : La chaîne de caractères où la représentation de l'entier sera stockée
> b : La base de la représentation (généralement 10 pour la base décimale, 16 pour la base hexadécimale, etc.)

```c
int main() {
    int n = 1234;  // L'entier à convertir
    char s[20];    // La chaîne de caractères qui
ecevra le résultat

    // Convertir l'entier en chaîne de caractères
en base 10
    itoa(n, s, 10);

    // Afficher le résultat
    printf("Le nombre %d en chaîne de caractères
est : %s\n", n, s);

    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
int main() {
    int n = 1234;  // L'entier à convertir
    char s[20];    // La chaîne de caractères qui
recevra le résultat

    // Convertir l'entier en chaîne de caractères en
base 2
    itoa(n, s, 2);

    // Afficher le résultat
    printf("Le nombre %d en binaire est : %s\n", n, s);
    return 0;
}
```

Remarque

La fonction **itoa()** n'est pas définie par la norme C et n'est pas disponible dans tous les environnements de programmation. Par exemple, elle n'est pas présente dans les environnements Unix/Linux. Dans ces cas, il est recommandé d'utiliser des alternatives standardisées, telles que **sprintf()** pour effectuer la conversion

```c
#include <stdio.h>

int main() {
    int n = 1234; // L'entier à convertir
    char s[20];   // La chaîne de
caractères qui recevra le résultat

    // Convertir l'entier en chaîne de
caractères en base 10
    sprintf(s, "%d", n);

    // Afficher le résultat
    printf("Le nombre %d en chaîne de
caractères est : %s\n", n, s);

    return 0;
}
```

ltoa (n_long, s, b) Convertit un long (n_long) en une chaîne de caractères.

> n_long : Le long à convertir
> s : La chaîne de caractères où la représentation du long sera stockée
> b : La base de la représentation

```c
int main() {
    long n = 1234567890L;  // Le long à convertir
    char s[20];            // La chaîne de caractères
qui recevra le résultat

    // Convertir le long en chaîne de caractères en
base 10
    ltoa(n, s, 10);

    // Afficher le résultat
    printf("Le nombre long %ld en chaine de caracteres
est : %s\n", n, s);

    return 0;
}
```

```
Le nombre long 1234567890 en chaine de caracteres est : 1234567890
```

6 Tableau de chaînes de caractères

- une chaîne de caractères est un tableau de caractères (type char), où chaque caractère est stocké dans un élément du tableau, et la chaîne se termine par un caractère spécial appelé le caractère nul ('\0').

- Un tableau de chaînes de caractères est un tableau qui contient plusieurs chaînes de caractères. Cela peut être vu comme une collection de chaînes, chaque chaîne étant un tableau de caractères.

A- Déclaration et initialisation

```
type nom[L][C];
```

```
char tableau_de_chaines[nb_de_chaines][longueur_max];
```

nb_de_chaines : nombre de chaînes dans le tableau

> **longueur_max : la longueur maximale de chaque chaîne, y compris le caractère nul ('\0')**

Exemple

```
char jour[7][9]={"lundi","mardi","mercredi","jeudi","vendredi","samedi","dimanche"};
```

Exemple : tableau de chaînes de caractères utilisé pour stocker les jours de la semaine

```c
#include <stdio.h>
int main() {
    // Déclaration d'un tableau de chaînes de
caractères représentant les jours de la semaine
    char jour[7][9] = {
        "lundi",     // "lundi" + 3 caractères nuls
        "mardi",     // "mardi" + 3 caractères nuls
        "mercredi",  // "mercredi" + 0 caractère nul
        "jeudi",     // "jeudi" + 3 caractères nuls
        "vendredi",  // "vendredi" + 0 caractère nul
        "samedi",    // "samedi" + 2 caractères nuls
        "dimanche"   // "dimanche" + 0 caractère nul
    };

    // Affichage des jours avec les caractères nuls
pour visualiser les chaînes
    for (int i = 0; i < 7; i++) {
        for (int j = 0; j < 9; j++) {
            printf("%c ", jour[i][j]); // Affichage de
chaque caractère
        }
        printf("\n");
    }

    return 0;
}
```

```
l u n d i \0 0 0 0
m a r d i \0 0 0 0
m e r c r e d i \0
j e u d i \0 0 0 0
v e n d r e d i \0
s a m e d i \0 0 0
d i m a n c h e \0
```

```
l u n d i
m a r d i
m e r c r e d i
j e u d i
v e n d r e d i
s a m e d i
d i m a n c h e
```

Le tableau jour est une matrice de :
7 lignes et **9 colonnes**

Chaque ligne représente un jour de la semaine, avec un maximum de **8 caractères** pour le nom du jour

B- Accès aux éléments d'un tableau de chaînes de caractères

L'accès aux éléments d'un tableau de chaînes de caractères en C peut être effectué à l'aide des indices de ligne et de colonne du tableau

Exemple

```c
#include <stdio.h>

int main() {
    char jours[7][9] = {
        "Lundi",
        "Mardi",
        "Mercredi",
        "Jeudi",
        "Vendredi",
        "Samedi",
        "Dimanche"
    };

    // Accès aux éléments individuels
    printf("Premier caractere du premier jour : %c\n", jours[0]
[0]); // L
    printf("Deuxieme caractere du deuxieme jour : %c\n", jours[1]
[1]); // a

    // Accès à une chaîne complète
    printf("Premier jour de la semaine : %s\n", jours[0]); // Lundi
    printf("Dernier jour de la semaine : %s\n", jours[6]); //
Dimanche
```

```
Premier caractere du premier jour : L
Deuxieme caractere du deuxieme jour : a
Premier jour de la semaine : Lundi
Dernier jour de la semaine : Dimanche
```

On peut aussi parcourir tout le tableau en utilisant une boucle :

Code

```c
#include <stdio.h>
int main() {
    char jours[7][9] = {
        "Lundi",
        "Mardi",
        "Mercredi",
        "Jeudi",
        "Vendredi",
        "Samedi",
        "Dimanche"
    };
    // Parcourir toutes les chaînes
    printf("Tous les jours de la semaine :\n");
    for (int i = 0; i < 7; i++) {
        printf("%s\n", jours[i]);
    }
    // Parcourir tous les caractères
    printf("\nTous les caracteres du tableau :\n");
    for (int i = 0; i < 7; i++) {
        for (int j = 0; j < 9; j++) {
            printf("%c", jours[i][j]);
        }
        printf("\n"); // Nouvelle ligne après chaque
chaîne
    }
    return 0;
}
```

Exécution

```
Tous les jours de la semaine :
Lundi
Mardi
Mercredi
Jeudi
Vendredi
Samedi
Dimanche

Tous les caracteres du tableau :
Lundi
Mardi
Mercredi
Jeudi
Vendredi
Samedi
Dimanche
```

```c
for (int i = 0; i < 7; i++) {   // Parcours chaque jour
    for (int j = 0; jours[i][j] != '\0'; j++) {  // Parcours
chaque caractère jusqu'au '\0'
        printf("%c", jours[i][j]);
    }
    printf("\n");  // Nouvelle ligne après chaque mot
}
```

VIIII – Les fonctions

Définition

On appelle fonction un sous-programme qui permet d'exécuter un ensemble d'instructions. Une fonction doit être déclarée, implémentée et enfin appelée soit par la fonction principale (main), soit par une autre fonction.

➡ Une fonction peut être appelée plusieurs fois à des endroits multiples du programme,

Déclaration

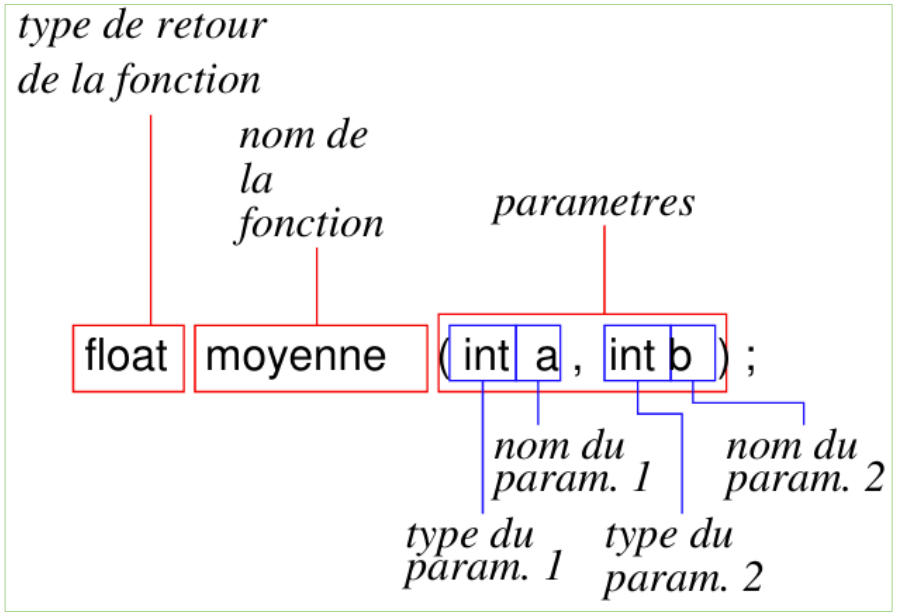
En langage C, une fonction doit être déclarée avant d'être utilisée. La déclaration (ou prototypage) définit son profil :

Type de retour : indique le type de la valeur renvoyée par la fonction. Si elle ne renvoie rien, on utilise **void**, et on parle alors de procédure.

Nom de la fonction : identifie la fonction.

Paramètres : une fonction peut prendre de zéro à n paramètres, chacun ayant un type et un nom.

Exemple



➡ Cette déclaration indique au compilateur que :
moyenne est une fonction,
Elle prend **deux paramètres de type int**,
Elle retourne une valeur **de type float** (un nombre réel).

Implémentation d'une fonction

Une fonction déclarée doit ensuite être implémentée. Il faut pour cela reprendre exactement la même signature et écrire le corps de la fonction.

➡ Une fonction peut être appelée plusieurs fois à des endroits multiples du programme,

Exemple

```c
#include <stdio.h>

// Déclaration de la fonction (prototypage)
float moyenne(int a, int b);

// Implémentation de la fonction
float moyenne(int a, int b) {
    float m;
    m = (a + b) / 2.0; // Assurer une division en float
    return m;
}

int main() {
    int x = 10, y = 20;
    printf("La moyenne de %d et %d est %.2f\n", x, y,
moyenne(x, y));
    return 0;
}
```

La déclaration de moyenne informe le compilateur avant son utilisation.

L'implémentation suit exactement la même signature et calcule la moyenne.

main() appelle la fonction et affiche le résultat.

Exécution

```
La moyenne de 10 et 20 est 15.00


...Program finished with exit code 0
Press ENTER to exit console.
```

Appel de fonction en langage C

L'appel d'une fonction permet d'exécuter le code qu'elle contient depuis n'importe quel endroit du programme. Une fonction peut être appelée

- dans **main()** (fonction principale)
- ou **dans une autre fonction** du programme.

Exemple

```c
#include <stdio.h>
// Déclaration des fonctions
float moyenne(int a, int b);
void afficherMoyenne(int x, int y);
// Implémentation de la fonction moyenne
float moyenne(int a, int b) {
    return (a + b) / 2.0;
}
// Implémentation de la fonction afficherMoyenne
void afficherMoyenne(int x, int y) {
    float m = moyenne(x, y); // Appel de la fonction moyenne
    printf("La moyenne de %d et %d est : %.2f\n", x, y, m);
}
int main() {
    int a = 10, b = 20;
// Appel de la fonction afficherMoyenne, qui elle-même appelle moye
    afficherMoyenne(a, b);
    return 0;
}
```

Exécution

```
La moyenne de 10 et 20 est : 15.00


...Program finished with exit code 0
Press ENTER to exit console.
```

Appel de fonction en langage C

En C, une fonction doit être déclarée avant son utilisation. Si on souhaite l'appeler dans main() (ou ailleurs), elle doit être déclarée au préalable, **sauf si un prototype est fourni au début du programme**. Cela permet au compilateur de connaître son type de retour et ses paramètres avant de l'utiliser.

Exemple : Utilisation d'un prototype avant main

```c
#include <stdio.h>

// Prototype de la fonction (déclaration)
float moyenne(int a, int b);

int main() {
    int a = 10, b = 20;
    printf("La moyenne de %d et %d est : %.2f\n", a, b, moyenne(a,
b));
    return 0;
}

// Définition de la fonction après main
float moyenne(int a, int b) {
    return (a + b) / 2.0;
}
```

Exécution

```
La moyenne de 10 et 20 est : 15.00


...Program finished with exit code 0
Press ENTER to exit console.
```

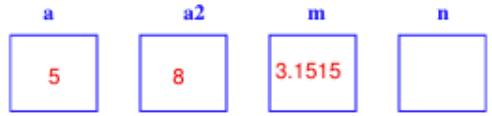
Portée des variables en C

Les **variables déclarées à l'intérieur d'une fonction** (y compris dans le prototype de la fonction) sont dites **locales à la fonction**. Cela signifie qu'elles **n'existent que dans cette fonction et ne peuvent pas être utilisées en dehors de celle-ci**. Chaque fonction possède son propre **espace mémoire** pour ses variables locales, ce qui permet d'utiliser le même nom de variable dans différentes fonctions sans conflit.

```c
#include<stdio.h>
float moyenne(int a,int b);
int main(){
float m ;
float n ;
int a ;
int a2;
a=5;
a2=8;
m=3.1515;
//instant(1)
n=moyenne (a,a2);
//instant(4)
printf("a=%d, a2=%d, m=%f, n=%f\n",a, a2,m,
n) ;
return 0;
}
float moyenne(int a,int b){
float m ;
int s;
//instant(2)
s=a+ b ;
m= s/2.0 ;
a=42;
//instant(3)
printf("a=%d, b=%d, m=%f,s=%d\n", a,b, m, s);
return m;
}
```

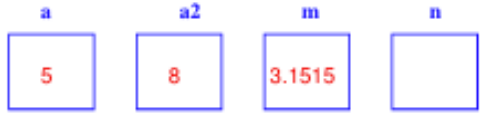
Etat de la mémoire à l'instant(1)

Dans la fonction **`main`**

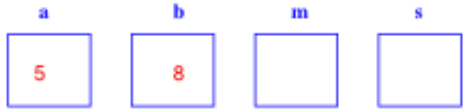


Etat de la mémoire à l'instant(2)

Dans la fonction **main**



Dans la fonction **moyenne**

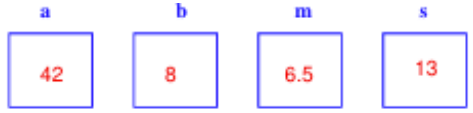


Etat de la mémoire à l'instant(3)

Dans la fonction **main**

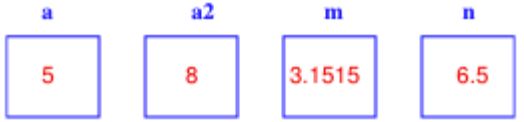


Dans la fonction **moyenne**



Etat de la mémoire à l'instant(4)

Dans la fonction **main**



Portée des variables en C

```c
#include<stdio.h>
float moyenne(int a,int b);
int main(){
float m ;
float n ;
int a ;
int a2;
a=5;
a2=8;
m=3.1515;
//instant(1)
n=moyenne (a,a2);
//instant(4)
printf("a=%d, a2=%d, m=%f, n=%f\n",a, a2,m,
n) ;
return 0;
}
float moyenne(int a,int b){
float m ;
int s;
//instant(2)
s=a+ b ;
m= s/2.0 ;
a=42;
//instant(3)
printf("a=%d, b=%d, m=%f,s=%d\n", a,b, m, s);
return m;
}
```

Etat de la mémoire aux instants 1, 2, 3 et 4

Instant 1 (Dans le main, avant l'appel de la fonction moyenne) :

- Dans la fonction main :
  - ➔ a = 5
  - ➔ a2 = 8
  - ➔ m = 3.1515
  - ➔ n = non initialisée

Instant 2 (Début de la fonction moyenne) :

- Dans la fonction main (ces valeurs sont conservées) :
  - → a = 5
  - → a2 = 8
  - → m = 3.1515
  - → n = non initialisée
- Dans la fonction moyenne :
  - → a = 5 (copie du paramètre)
  - → b = 8 (copie du paramètre)
  - → m = non initialisée
  - → s = non initialisée

Instant 3 (Dans la fonction moyenne, après les calculs) :

- Dans la fonction main (inchangé) :
  - → a = 5
  - → a2 = 8
  - → m = 3.1515
  - → n = non initialisée
- Dans la fonction moyenne :
  - → a = 42 (modifié par a=42)
  - → b = 8 (inchangé)
  - → m = 6.5 (calculé par s/2.0)
  - → s = 13 (calculé par a+b)

Instant 4 (Retour dans le main) :

- Dans la fonction main :
  - → a = 5 (inchangé car la modification de a dans moyenne n'affecte pas le main)
  - → a2 = 8 (inchangé)
  - → m = 3.1515 (inchangé)
  - → n = 6.5 (reçoit la valeur retournée par moyenne)

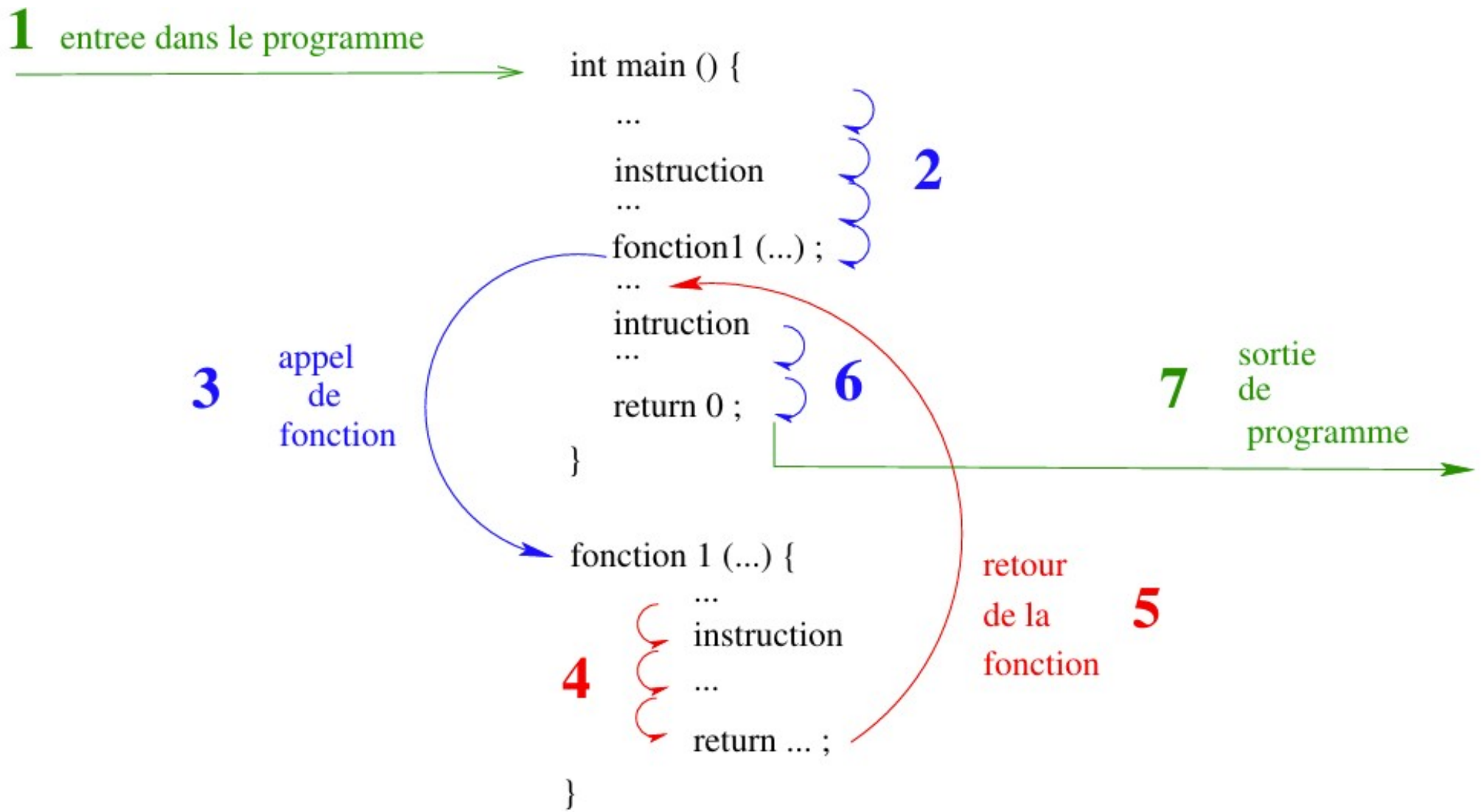
Points importants à noter :

1- La modification de **'a'** dans la fonction moyenne **(a=42)** n'affecte pas la variable **'a'** du main car les paramètres sont passés par valeur.

2- Chaque fonction a son propre espace mémoire pour ses variables locales.

3- Les variables locales de la fonction moyenne (m et s) ne sont plus accessibles après la fin de la fonction.

Lorsque la fonction se termine, l'exécution du programme reprend immédiatement après l'endroit où la fonction a été appelée. Ce processus est appelé **retour de fonction**.



Points importants à respecter :

1- **Cohérence des signatures** :
→ La déclaration (prototype) et l'implémentation doivent être identiques
  → Types des paramètres
  → Type de retour
  → Nombre de paramètres

2- **Règles de déclaration** :
→ Toute fonction doit être déclarée avant son utilisation
→ La déclaration peut se faire soit par prototype, soit par implémentation directe

3- **Règles d'implémentation** :
→ Chaque fonction déclarée doit être implémentée
→ Le type de retour doit correspondre à la déclaration
→ Les paramètres doivent correspondre en type et en nombre

4- **Règles d'appel** :
→ Les arguments passés doivent correspondre aux types attendus
→ Le nombre d'arguments doit être exact
→ La valeur retournée doit être utilisée de manière cohérente avec son type

Exercice d'application

Ecrire une fonction **puissance**, qui prend deux paramètres entiers (la valeur et l'exposant), puis calcule et renvoie la valeur à la puissance l'exposant. Par exemple, puissance(2, 3) retourne 8.

Testez la fonction en écrivant la fonction **main** qui :
A- demande à l'utilisateur la valeur et l'exposant
B- appelle la fonction puissance ainsi écrite.

```c
#include <stdio.h>
 //#include <math.h>
 int puissance(int valeur,int exposant) {
 int resultat= 1 ;
 int i ;
 for (i=0;i<exposant;i++){
 resultat= resultat*valeur ;
 }
 return resultat;
 }
 int main() {
 int v ;
 int e ;
 int resultat;
 printf("rentrez une valeur:") ;
 scanf ("%d",&v) ;
 printf("rentrez l'exposant:") ;
 scanf ("%d",&e) ;
 resultat=puissance(v, e);
 printf("%d puissance %d vaut %d\n", v, e, resultat);
 return 0;
 }
```

Exemple de déroulement des itérations pour 2 puissance 3

Itération (i)	Avant itération (resultat)	Multiplication (resultat = resultat * valeur)	Après itération (resultat)
i = 0	1	1 * 2	2
i = 1	2	2 * 2	4
i = 2	4	4 * 2	8

En utilisant la fonction **pow()** de <math.h>

```c
#include <stdio.h>
#include <math.h>
int main() {
    int v;
    int e;
    double resultat;  // On utilise double car pow() renvoie un double
    printf("rentrez une valeur: ");
    scanf("%d", &v);
    printf("rentrez l'exposant: ");
    scanf("%d", &e);

    resultat = pow(v, e);
    printf("%d puissance %d vaut %.0f\n", v, e, resultat);

    return 0;
}
```